Computers, Materials & Continua

DOI:xxxxx

Type: XXXX

A Review of AI-Driven Automation Technologies: Latest Taxonomies, Existing Challenges, and Future Prospects

Weiqiang Jin [1,2] Ningwei Wang[1], Lei Zhang[3], Xingwu Tian[1], Bohang Shi[1], and Biao Zhao[1,*]

Received: Month Day, Year; Accepted: Month Day, Year.

[1]School of Information and Communications Engineering, Xi'an Jiaotong University, iHarbour, Xi'an, Shanxi, 710049, China.

[2]Department of Electrical and Electronic Engineering, The University of Hong Kong, Pokfulam Road, Hong Kong, 999077, China.

[3]School of Intelligence Science and Technology, Beijing University of Civil Engineering and Architecture, Beijing 102616, China.

[*]Corresponding Authors: Biao Zhao. Email: biaozhao@xjtu.edu.cn.

Abstract:

With the growing adoption of Artifical Intelligence (AI), AI-driven autonomous techniques and automation systems have seen widespread applications, become pivotal in enhancing operational efficiency and task automation across various aspects of human living. Over the past decade, AI-driven automation has advanced from simple rule-based systems to sophisticated multi-agent hybrid architectures. These technologies not only increase productivity but also enable more scalable and adaptable solutions, proving particularly beneficial in industries such as healthcare, finance, and customer service. However, the absence of a unified review for categorization, benchmarking, and ethical risk assessment hinders the AI-driven automation progress. To bridge this gap, in this survey, we present a comprehensive taxonomy of AI-driven automation methods and analyze recent advancements. We present a comparative analysis of performance metrics between production environments and industrial applications, along with an examination of cutting-edge developments. Specifically, we present a comparative analysis of the performance across various aspects in different industries, offering valuable insights for researchers to select the most suitable approaches for specific applications. Additionally, we also review multiple existing mainstream AI-driven automation applications in detail, highlighting their strengths and limitations. Finally, we outline open research challenges and suggest future directions to address the challenges of AI adoption while maximizing its potential in real-world AI-driven automation applications.

Keywords: AI-driven Automation Techniques and Systems, Artificial General Intelligence (AGI), LLMs, Robotic Process Automation (RPA)

1 Introduction

1.1 Research Backgrounds of AI-Driven Automation

The evolution of artificial intelligence (AI) [1, 2, 3] technologies has advanced significantly in recent years, bringing about a new era of AI-driven autonomous tools and automation system [4, 5, 6, 7]. Numerous advanced AI-driven automation technologies [8, 9, 10, 11] have been developed to address specific challenges within various aspects of human production and daily life. These advancements have enabled organizations to

This work is licensed under a Creative Commons Attribution 4.0 International License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

optimize operational efficiency, reduce costs, and enhance productivity across a variety of practical application scenarios, from healthcare [12, 13, 14] to finance [15, 16, 17]. AI-driven automation systems [18, 19, 20] are now capable of automating repetitive and time-consuming tasks, allowing human workers to focus on more efficient activities such as strategic planning and creative problem solving [21, 22, 23].

1.1.1 Advancements of AI-Driven Automation

Specifically, AI-driven automation encompasses a wide range of technologies [6, 9, 24, 25], which together streamline complex operations, enhance task-specific decision-making capabilities, and significantly boost practical operational efficiency. Among the various approaches to AI-driven automation, several stand out as particularly influential in driving the field forward, including Robotic Process Automation (RPA) [14, 26, 27], No-code platforms [28, 29, 30], and Large Language Models (LLMs) [31, 32].

(1). Firstly, Robotic Process Automation (RPA) [26, 33] is a key technique in AI-driven automation, utilizing software robots to perform repetitive tasks traditionally handled by humans. RPA is particularly prevalent in industries such as finance, healthcare, and manufacturing, where efficiency and error reduction are critical [34, 35].

(2). Meanwhile, No-code platforms [28, 36, 37, 38] are another important approach in AI-driven automation, further broadening the accessibility of automation technologies. They enable users without technical expertise to design complex workflows and automation systems, democratizing the development of AI solutions. With intuitive interfaces that allow for the creation of complex workflows without requiring coding skills, these platforms empower users to build customized automation solutions [39, 40, 41, 42].

(3). Lastly, LLMs [43, 44, 45] have achieved exceptional success across a wide range of tasks, from natural language processing (NLP) [46, 47] to decision support systems [48, 49, 50]. By utilizing vast amounts of textual data, these models generate accurate and contextually relevant outputs, making them indispensable in applications such as chatbots [51, 52], document automation [53, 54, 55, 56], and customer service [10, 57, 58].

In recent years, among them, the most transformative advances in AI-driven automation is the integration of LLMs into task-execution and production processes [59, 56, 60]. The integration of LLMs and traditional workflow automation techniques dramatically improves the effectiveness and efficiency of human production processes such as customer service [10, 57, 58], document processing [2, 53, 54], and decision-making support [61, 62, 63], allowing people and organizations to develop more intelligent systems capable of understanding complex tasks and delivering optimized solutions.

Overall, AI-driven automation has quickly become a transformative force across various industries, offering tremendous potential in improving efficiency, accuracy, and scalability.

1.1.2 Challenges in AI-Driven Automation

However, as the technology continues to evolve, AI-driven automation faces several significant technical challenges that hinder its widespread adoption and effective implementation [56, 64, 65]. These challenges, ranging from data governance [2, 54, 66] to model interpretability [13, 64, 67] and deployment complexity [20, 55, 56, 68], present barriers that need to be addressed to fully realize the potential of AI-driven automation.

Specifically, AI-driven automation faces multiple technical hurdles: (1). Data quality and governance issues [35, 69, 70], including noise, privacy, and compliance; (2). Model interpretability gaps in "black-box" systems [64, 67, 71], critical for high-stakes domains; (3). Generalization limitations in unseen scenarios [72, 73, 74]; (4). Scalability demands [66, 75, 76] for large-scale, low-latency deployment; (5). Algorithmic bias perpetuating unfair outcomes [77, 78, 79]; (6). Integration complexity with legacy systems [14, 17, 80]; (7). Real-time processing requirements [18, 81, 82]; and (8) Security vulnerabilities to adversarial attacks

[58, 73, 76].

Addressing these issues is essential for advancing the AI-driven automation research. Overcoming these challenges will enable AI-driven automation to reach its full potential, leading to more efficient and effective systems across industries.

1.2 Motivations of This Work and Our Contributions

While numerous innovative AI automation-related works, particularly with the advanced LLMs [77, 83, 84] and Robotic Process Automation (RPA) [26, 54, 82], have introduced various novel technical concepts and knowledge in the research of AI-driven automation. Despite these successes and increasing attention, a comprehensive review of the current AI-driven automation developments still remains limited [75, 77, 85, 86]. Moreover, these latest developments have introduced many innovative framework and approaches that were not covered in earlier literature [67, 75, 86], highlighting the need for more updated and structured overview and arrangement for these literature.

Given the accelerating technology progress in AI-driven automation [6, 25, 70, 76], in this paper, we are motivated to present a novel taxonomy that categorizes and unifies the various methods, tools, and applications. We believe that this will help subsequent researchers with a better and clearer understanding of the key technical features and workflows of AI-driven automation [17, 75]. We believe this work will contribute to the development of AI-driven automation, offering insights into how different components/modules interact and drive overall performance [69, 82]. This review aims to help researchers identify emerging trends, and highlight critical gaps that require further exploration to advance the frontiers of AI-driven automation.

In summary, in this survey, we make the following key contributions:

[1]. Firstly, we propose a novel taxonomy to classify the different methods used in AI-driven automation. This different taxonomy provide clarity on the strengths and weaknesses of each categories and how they can be applied in various practical application scenarios.

[2]. Then, we also systematically evaluate current mainstream tools and systems used for AI-driven automation, discussing their strengths and limitations, and offering insights into the kinds of approaches.

[3]. Finally, we discuss the major challenges facing the field, including data privacy, ethical concerns, and the lack of regulatory frameworks, and propose strategies for overcoming these obstacles.

[4]. Meanwhile, we suggest promising areas for future research of AI-driven automation, particularly in improving LLM transparency, developing more efficient RPA techniques, and addressing regulatory challenges in the AI space.

1.3 Methodology for Literature Selection

To ensure a comprehensive and unbiased selection of relevant literature, we employed a systematic search strategy across multiple academic databases, including IEEE Xplore[3], ACM Digital Library[4], ScienceDirect[5], SpringerLink[6], and arXiv[7]. The search query was designed to capture key concepts related to AI-driven automation tools, their functionalities, and applications.

Specifically, our survey adopts a systematic approach to literature selection, prioritizing peer-reviewed journal articles, conference proceedings, and high-impact preprints We focus exclusively on publications that address AI-driven automation tools, their technical architectures, and real-world applications, ensuring alignment with our taxonomy development goals. The selected works were published between 2010 and

[3]IEEE Xplore: https://ieeexplore.ieee.org/Xplore/home.jsp.
[4]ACM Digital: https://dl.acm.org/.
[5]ScienceDirect: https://www.sciencedirect.com/.
[6]Springer Nature Link: https://link.springer.com/.
[7]arXiv: https://arxiv.org/.

2025, a period that captures both foundational advancements in AI-automation techniques [26, 27, 62] This timeframe was chosen to reflect the rapid evolution of the field, including breakthroughs such as transformer-based models (e.g., GPT-4 [45]).

The initial screening of titles and abstracts refined the candidate pool from over 3000 articles to the most relevant works. Among over 250 core references selected, approximately 10% originate from 2010 to 2018, representing seminal works on RPA infrastructure and early NLP integration. The period 2019 to 2021 accounts for 35% of references, coinciding with the emergence of LLM-powered automation frameworks. The majority at 55% derive from 2022 to 2025. This ensures balanced coverage of historical foundations and representative advancements while minimizing recency bias through rigorous citation analysis and validation.

1.4 Survey Roadmap

This survey is systematically organized to provide a comprehensive analysis of AI-driven automation tools, their challenges, and future directions. The structure is as follows: Section 1 introduces the transformative impact of AI-driven automation across industries, outlines the scope of this survey, and presents its organizational framework. Section 2 establishes systematic taxonomies to classify AI-driven automation tools into four architectural paradigms: rule-based, LLM-driven, multi-agent cooperative, and hybrid models, further categorizing them by functionality, execution mechanisms, and human-AI collaboration modes. Section 3 conducts a comparative analysis of state-of-the-art tools (e.g., Manus, OWL, LangChain, UiPath), emphasizing their applicability in domains like healthcare, finance, and manufacturing. Section 4 examines persistent challenges, including data governance gaps, model interpretability deficits, and scalability bottlenecks, while proposing mitigation strategies. Section 5 explores emerging trends (e.g., self-debugging workflows, AGI integration, and quantum-resistant security) and forecasts future research trajectories. Section 6 concludes by synthesizing key insights and advocating for interdisciplinary collaboration to align technological advancements with ethical and societal values.

2 A Comprehensive Technological Taxonomy of AI-Driven Automation

Recent advances of *Machine Learning* (ML) [87, 88, 89] and *Artificial Intelligence* (AI) [72, 78, 83, 90] in have led to a proliferation of AI-driven automation tools, each designed to address specific challenges in workflow optimization, decision-making, and task execution [91, 92, 93, 94]. As these tools grow increasingly diverse in their architectures, functionalities, and applications, there is a pressing need for a systematic taxonomy to categorize and compare their capabilities [95, 96, 97, 98]. Currently, the lack of a unified framework makes it difficult for researchers and practitioners to evaluate the suitability of these tools for different use cases [99, 100, 101, 102]. Thus, this section aims to address this gap by proposing several structured classifications from different dimensions of various AI-driven automation tools, analyzing their key characteristics, and highlighting their roles in modern automation ecosystems.

As illustrated in Figure 1, these tools exhibit diverse architectural paradigms and functional specializations. The proposed taxonomy systematically organizes these systems across five key dimensions: (1) architectural foundations (Section 2.1), (2) core functionalities (Section 2.2), (3) task flow management (Section 2.3), and (4) application domains (Section 2.4).

2.1 Classification on Architecture of AI-Driven Automation Tools

The architectural landscape of modern AI-driven automation tools can be categorized into four distinct paradigms, including Rule-based [2, 15, 103, 54], LLM-based [59, 72, 78, 87], Multi-Agent-based [48, 91, 92, 93] and Hybrid Models [7, 33, 55, 86], each addressing specific operational requirements and complexity levels [6, 96, 97, 98]. The architecture classification of automation technologies is outlined in Table 1.

Specifically, 1. *Rule-Based Systems* [11, 103, 54] represent the foundational layer of automation architectures, employing deterministic logic through predefined condition-action rules. 2. *LLM-Based*

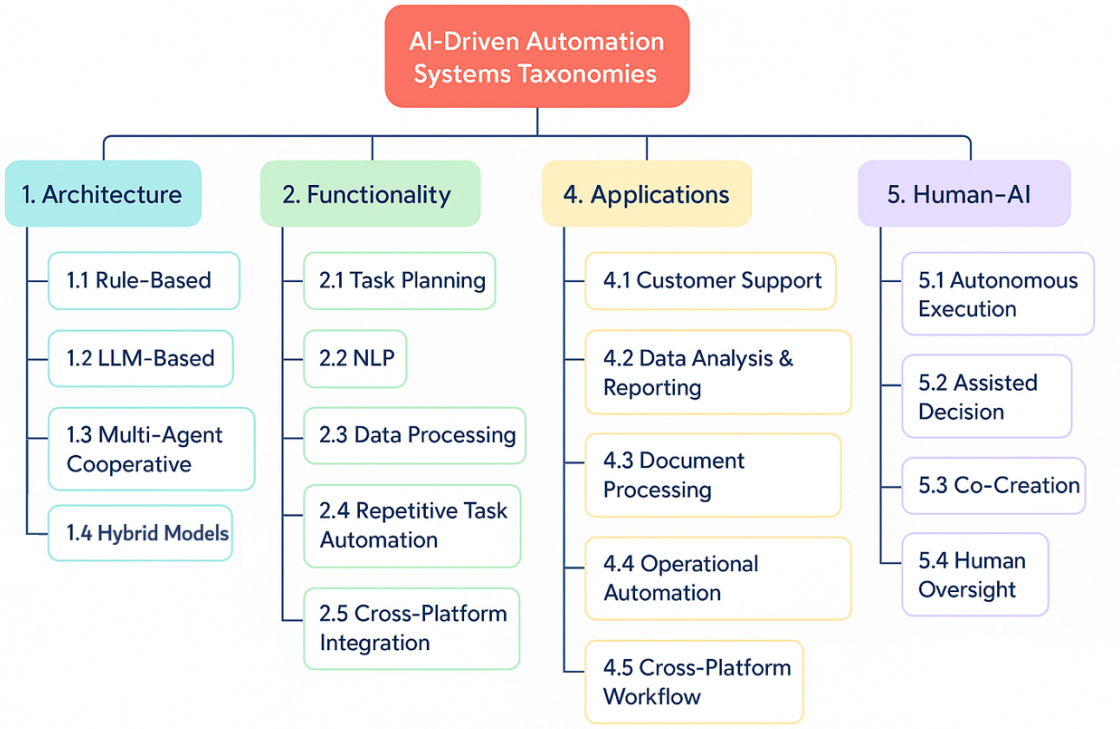


Figure 1: The Taxonomy Visualization of Architectural Approaches for AI-driven Automation.

Systems [46, 77, 87, 124] mark a paradigm shift toward cognitive automation, leveraging LLMs to process unstructured data and execute language-centric tasks. 3. *Multi-Agent Systems* [125, 126, 127] introduce distributed intelligence through coordinated autonomous agents, as implemented in platforms like Manus [114, 117, 119] and OWL [120]. 4. *Hybrid Models* [7, 33, 55] combine the strengths of rule-based and AI-driven approaches, exemplified by UiPath AI Fabric's integration of RPA with machine learning.

Figure 2 illustrates the evolutionary landscape of automation system architectures over time. It highlights four major phases, Rule-Based, Multi-Agent, LLM-Based, and Hybrid Agentic, alongside their timeframes and defining characteristics [91, 94, 95, 96]. This shows how each paradigm emerged, developed, and influenced the next, reflecting shifts in both technical capability and system design philosophy. Notably, these paradigms are not isolated; rather, they exhibit significant overlaps and integration points, emphasizing that no single approach exists independently of the others in practice [5, 6, 75].

As shown in Figure 3 (1), this is a typical structure of rule-based automation systems [11, 51, 105]. This architecture is centered around deterministic processing driven by predefined logic. The system accepts structured and pre-validated input from the user or an external source. These inputs are fed into a logic engine, which applies rule-based conditions-often expressed in IF-THEN format-for pattern matching [108, 110]. Each rule is associated with a deterministic action, such as sending an alert or executing an API call [59, 80, 128].

The rule base in this like automation system is a separate, static, and version-controlled module, decoupled from the logic execution layer [51, 108]. This ensures maintainability and traceability of the rule set over time. The outputs are standardized and include actionable signals like alerts or service calls. The model emphasizes the non-probabilistic, mechanical nature of such systems, with fixed logical flows and strict dependency on exact condition matching. This kind of architecture is highly interpretable, though it requires manual effort to maintain and update the rule repository [109, 137].

Figure 3 (2) illustrates a typical architecture prototype of a LLM-based automation system. The

Table 1: Categories Divided by Architecture of AI-Driven Automation Tools

AI-Driven Automation Categories	Descriptions	Examples
Rule-Based	These tools rely on pre-defined rules and logic to perform tasks. The rules dictate actions based on conditions.	UiPath [104], ChatbotRev [105] [106, 107] HeyTAP [108], ChatIoT [109], RecRules [110]
LLM-based	These tools use LLMs to understand and generate natural language, automating tasks related to text and decision-making.	LangChain [10, 111, 112], Auto -GPT [113], GPT Researcher [44], OpenManus [114], BabyAGI [115], AgentGPT [31], SuperAGI [116]
Multi-Agent Cooperative	These systems use multiple autonomous agents to collaboratively work on tasks, with task decomposition and coordination between agents.	Manus [117, 118, 119], OWL [120] [120], MetaGPT [60], AutoGen [91], AgentVerse [65], CrewAI [121], ChatDev [47], TradingGPT [16]
Hybrid Models	Combining RPA, AI, and machine learning models, hybrid tools use both pre-programmed rules and AI decision-making for task execution.	UiPath AI Fabric [104, 106] [107], AirSlate [122], Zapier [30], SmartFlow [55], LMRPA [33], AutoAgent [123], EconAgent [32]

architecture adopts a modular design that clearly delineates core processing flows and external interaction interfaces, enabling an end-to-end intelligent mapping from input to output.

Firstly, the *input module* receives natural language requests or structured data from the user and forwards them to the *Context Manager*, which maintains dialogue history, task context, and multi-turn interaction states-essential for preserving semantic continuity and task awareness [70]. Next, the *Memory Module* organizes both short-term and long-term information, allowing fast retrieval of contextual data and knowledge fragments. On top of this, the system can integrate *Retrieval-Augmented Generation* (RAG) [46, 138, 139, 140], leveraging external vector databases for semantic search to enhance the model's knowledge base. The *Reasoning Module* is responsible for understanding intent, making logical decisions, and invoking tools [5, 141, 142]. It interacts with *External APIs* to query databases or perform calculations. Finally, outputs are then passed through *Rule Guardrails* [105, 109] to ensure compliance with business rules, ethical standards, and formatting constraints. Finally, the *Post-Processor Module* [102, 143] sanitizes the raw output, applies safety filters, and formats the response before it is delivered to the user through the *output module*, ensuring clarity and reliability.

Figure 3 (3) presents a hierarchical architecture for multi-agent automation systems that integrates dynamic task decomposition with a Mixture-of-Experts (MoE) [131, 132] execution layer. The system begins with a main task input, which is parsed by a Task Decomposer module responsible for dividing the task into coherent subtasks based on task semantics and dependency graphs. These subtasks are then routed to an MoE pool containing specialized expert agents, such as reasoning experts [61, 78, 90], data processing agents [53, 144, 145], and API orchestration agents [59, 79, 80, 128].

Routing decisions are made based on task embeddings and gating networks that dynamically assign subtasks to the most suitable experts [131, 146, 147, 148]. Subtasks are executed in parallel, and their results are aggregated into a unified output. Conflict resolution and validation mechanisms ensure consistency and robustness. This architecture supports modular scalability, fault tolerance, and efficient parallelism, making it well-suited for complex, heterogeneous, and cross-domain automation scenarios [39, 55, 91, 94].

Figure 3 (4) depicts a hybrid model-based automation architecture that combines rule-based systems with AI models to achieve adaptive and resilient decision-making [7, 33, 86, 129]. The system begins by routing user input-either structured or unstructured-through a decision router. Structured data is directed to a deterministic rule engine, while unstructured or context-dependent input is processed by an AI model [59, 72, 78, 90]. The outputs from both branches are validated through a result validation module that ensures consistency and accuracy.

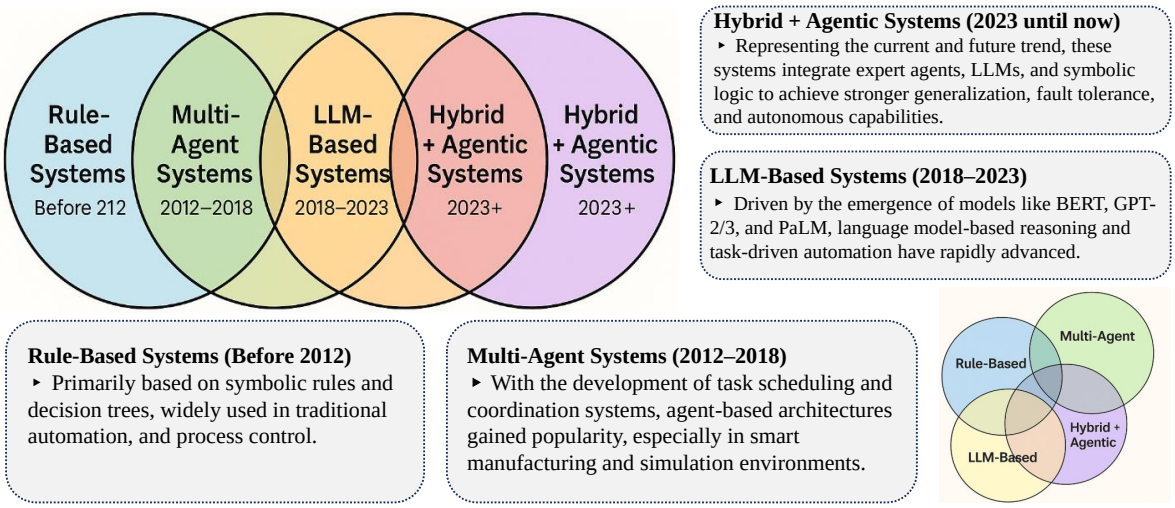


Figure 2: The Evolutionary Trajectory and Integration of Automation Architectures: *From Rule-Based Systems to Hybrid Agentic Intelligence*.

Depending on the decision type (probabilistic or deterministic), results are either accepted or passed to a fallback handler when anomalies or inconsistencies are detected. The fallback mechanism includes model switching, rule simplification, or escalation to human review [7, 83, 98]. A knowledge graph [149] supports both the router and AI model in contextual understanding, and external retrieval modules assist in enriching or verifying the input [132, 144, 145, 150]. This hybrid approach balances the explainability and speed of symbolic logic with the flexibility and adaptability of neural models, making it suitable for high-stakes or regulated automation domains [33, 55, 86, 129].

Furthermore, Figure 4 presents a comparative architecture overview of four representative categories of AI-driven automation system. The figure is designed to highlight the core structural patterns and processing flows underlying each class, including rule-based systems, multi-agent automation systems, LLMs-based decision frameworks, and hybrid-AI integrated models [91, 94, 95, 96]. The visualization contextualize the diversity of design philosophies and operational mechanisms across these automation systems [39, 47, 55, 151]. This comparison supports the analysis of trade-offs in scalability, interpretability, robustness, and domain alignment across different automation paradigms.

Overall, the architectural progression from rule-based systems to hybrid systems and LLM-based systems reflects an industry-wide transition from mechanical task replication to cognitive process optimization [54, 72, 78, 87]. Current research focuses on developing systems that self-adaptive dynamically select appropriate architectural paradigms based on real-time workflow analysis [99, 97, 146, 147].

2.2 Classification on Functionality of AI-Driven Automation Tools

The functional landscape of AI-driven automation tools can be systematically organized into five principal categories, including *Task Planning, NLP, Data Processing, Repetitive Task Automation*, and *Cross-Platform Integration* [5, 6, 95]. Each represents distinct technical approaches to workflow automation [69, 75]. This classification, presented in Table 2, reveals the evolving capabilities of modern automation systems from isolated task execution to integrated cognitive platforms.

As shown in Figure 5 (1), **Task Planning Architecture** enables intelligent workflow automation through dynamic goal decomposition and adaptive execution. The system first parses natural language objectives into structured representations using LLMs. A Graph Neural Network (GNN) [164] then hierarchically decomposes tasks while modeling dependencies between sub-tasks as directed edges [165, 166]. The

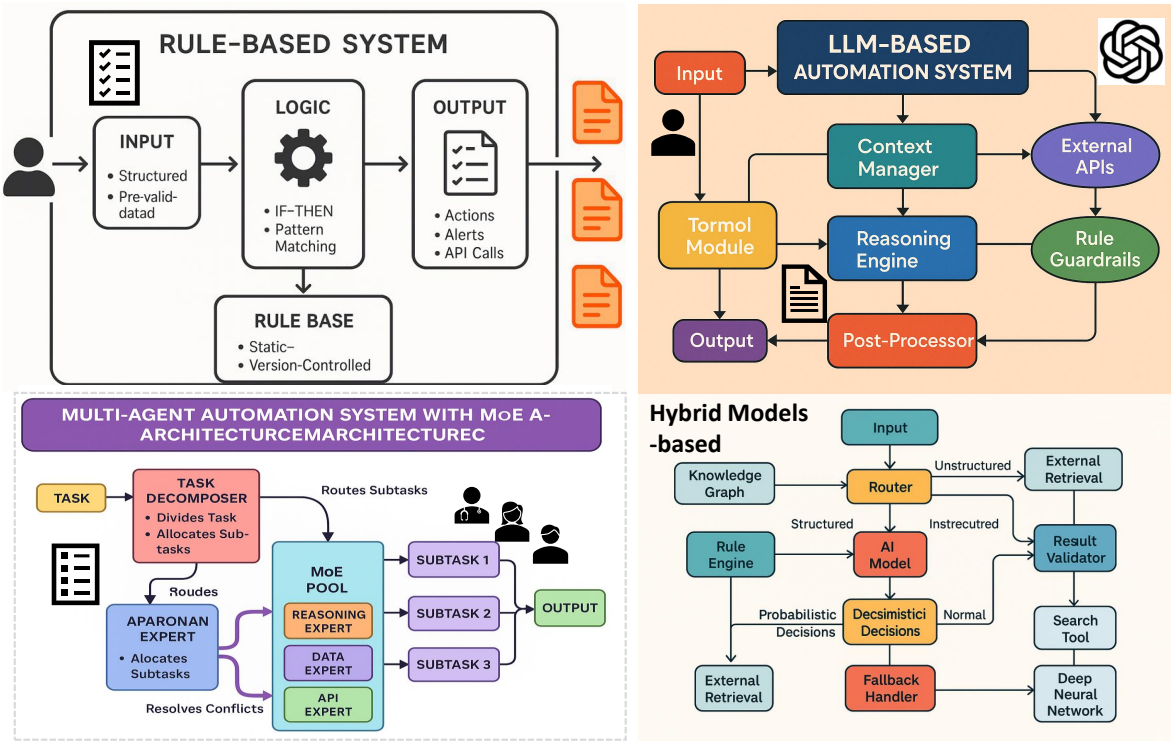


Figure 3: The representative architectural prototypes of four major categories of existing automation systems: (1) Rule-based systems [105, 109, 110], (2) Multi-agent systems [65, 129, 130] with Mixture-of-Experts (MoE) [131, 132], (3) Multi-agent-based automation decision architectures [133, 134, 135, 136], and (4) Hybrid models [7, 33, 55] integrating symbolic rules with AI reasoning and fallback mechanisms.

architecture features three innovation layers: (1) *Resource-Aware Allocation* that dynamically assigns compute resources (GPU/CPU) based on sub-task requirements [25, 27, 167], (2) *Reinforcement Learning Scheduler* optimizing execution order through real-time performance feedback [16, 85, 168], and (3) *Fault-Tolerant Monitoring* triggering granularity adjustments when success rates drop below thresholds [39, 56, 136].

As depicted in Figure 5 (2), **NLP Processing-based Automation** enables multimodal automation through layered linguistic analysis [79, 149, 169, 170]. The raw inputs (text/voice/image) undergo unified preprocessing via *SentencePiece* tokenization [171], *Whisper* speech recognition [160, 161], or *CLIP* visual encoding [162]. The system then processes embeddings through 12-layer GPT-4 transformer blocks [156, 43], generating parallel outputs for: (1) *Intent Classification* using few-shot learning [66, 172], (2) *Entity Recognition* with domain-enhanced BIO tagging [170], and (3) *Aspect-based Sentiment Analysis* via VADER++ [169, 173]. These outputs converge at the *Action Engine*, which validates parameters against API schemas (e.g., mapping product mentions to CRM IDs) and triggers downstream workflows [91, 92, 174].

As illustrated in Figure 5 (3), **Cognitive Data Processing**-based automation system demonstrates a comprehensive architecture for intelligent data automation. The pipeline initiates with *Raw Data Sources* spanning structured databases (SQL/NoSQL), semi-structured documents (PDF/Excel), and real-time IoT streams [14, 34, 55, 66]. The cognitive data processing pipeline features three core processing stages: (1) *Smart Connectors* [80, 81, 82] that perform schema autodiscovery and adaptive parsing of diverse data sources, (2) *Automated ETL* [17, 69, 75] with dual-phase processing including data cleansing (missing value imputation, outlier correction) and feature engineering (temporal patterns, embeddings), and (3) *Feature Store* [14, 34, 76] that maintains versioned features with complete lineage tracking.

As illustrated in Figure 5 (4), **Repetitive Task Automation** systems combine rule-based execution with

Dimension	Rule-Based Systems	Multi-Agent Systems	LLM-Based Systems	Hybrid Models
Core Principle	Predefined condition-action rules	Distributed expert agent collaboration	Large language model semantic understanding	Dynamic routing (rules + AI)
Processing Speed	Ultra-fast (μs-level)	Moderate (network overhead)	Slow (100ms~s-level)	Fast (rule-prioritized)
Scenario Adaptability	Structured scenarios only	Cross-domain complex tasks	Open-domain tasks	Domain-adaptive (hybrid decision)
Error Handling	Predefined fallback	Inter-agent negotiation	Generative solutions	Dual-validation fallback
Scalability	Manual rule addition	Dynamic agent hot-swapping	API plug-and-play	Synchronized rule/model updates
Interpretability	Fully transparent	Local (per-agent) explainability	Black-box (requires LIME/SHAP)	Partial transparency
Typical Use Cases	• Financial risk control • Industrial pipelines	• Supply chain optimization • Smart city scheduling	• Customer service chat • Document generation	• Medical diagnosis • Insurance claims
Maintenance Cost	Low	Moderate	Very high (compute/data)	High (dual maintenance)
Key Technologies	• Decision trees • State machines	• MoE routing • DAG task decomposition	• Prompt engineering • RAG	• Confidence-based routing • Rule distillation
Compliance Risk	Low (deterministic)	Medium (agent reliability)	High (uncontrolled generation)	Minimal (dual-check)

Figure 4: A Comprehensive Framework Comparison of AI-Driven Automation Systems.

adaptive computer vision (CV) [49, 50] and natural language processing (NLP) [46, 148, 175, 176] to handle high-volume, predictable workflows. The architecture follows a cyclical pattern where tasks are: (1) triggered by scheduled events or system alerts, (2) processed through standardized rules with contextual adaptations, and (3) executed with consistent output formats [33, 177, 178]. Key innovations include *vision-enabled element recognition* that maintains action precision across interface changes, and *performance auto-tuning* that optimizes processing time while preserving output consistency. These systems excel in scenarios like invoice processing (extracting fixed fields from varying layouts) or data migration (repetitive database operations).

Figure 5 (5) illustrates a generalized architecture for **Cross-Platform Automation Systems**, designed to bridge heterogeneous data environments through intelligent integration layers [65, 130, 179]. The system begins with diverse *Source Systems* such as SaaS applications, legacy databases, and IoT devices [70, 76, 180]. A *Neural API Mapper* decodes protocol formats, aligns schemas using graph neural networks, and securely translates authentication protocols (e.g., OAuth2 [181, 182] to SAML [143, 183]) . The *Adaptive Transformer* module converts between data formats (e.g., JSON, XML, Protobuf) and ensures quality-of-service via retry logic and rate limiting. Processed data flows into *Target Systems*, including cloud platforms and mobile endpoints, while a *Monitoring Hub* tracks API calls, schema drift, and latency metrics [184, 185, 186]. This architecture emphasizes modularity, protocol adaptability, and real-time observability, making it particularly suitable for hybrid cloud integration and dynamic enterprise workflows.

Beyond the functional taxonomy, understanding the relative maturity of these approaches is crucial for researchers alike. Figure 6 presents a technology maturity curve analysis for the five functional categories, synthesizing insights from current implementations, representative systems, and ongoing research challenges. This indicates that while AI-enhanced repetitive task automation is nearing mainstream adoption, and cross-platform integration tools are demonstrating increasing robustness, the more cognitive capabilities, particularly NLP integration, cognitive data processing, and advanced task planning, exhibit significant potential but face longer development paths towards stable, widespread deployment. NLP integration shows rapid progress but requires refinement in operational reliability within workflows. Cognitive data processing and task planning represent the frontier of AI-driven automation, currently characterized by innovative prototypes and active research, with substantial work needed to overcome challenges in scalability, robustness, and generalization for complex, real-world environments.

Table 2: Functional Classification of AI-Driven Automation Tools

Functionality	Technical Characteristics	Representative Systems
Task Planning	Hierarchical task decomposition using graph neural networks.	Manus [117, 118, 119], OpenManus [114], LangChain [10, 111, 152, 153], ToRA [154], AutoAgent [123], TaskMatrix.AI [23], LLM-Planner [155]
Natural Language Processing (NLP)	Transformer-based semantic parsing and multimodal interaction.	GPT-4 [43, 156, 45], DeepAI [157], Hugging Face [158, 159], MM-REACT [79], Whisper [160, 161], CLIP [162], airSlate [122], LLaMA [88, 89], Docling [53]
Cognitive Data Processing	Cognitive ETL pipelines with automated feature engineering.	MetaGPT [60], OWL [120], UiPath AI Fabric [106, 107]
Repetitive Task Automation	Vision-enabled RPA with contextual adaptation.	UiPath [106], Zapier [30], AirSlate [122] SmartFlow [55], LMRPA [33], BreachSeek [163]
Cross-Platform Integration	Neural API mapping and protocol translation.	OWL [120], UiPath [106], Zapier [30], ToolLLM [59], AutoGen [91], AgentVerse [65], CrewAI [121], Airtable [28], Voiceflow [37]

Overall, the functional evolution of AI-driven automation tools reveals three significant technical trends: the convergence of symbolic and connectionist AI approaches [78, 96, 142], the emergence of self-configuring automation ecosystems [92, 94, 187], and the growing importance of human-AI collaboration frameworks [188, 189, 190]. Current research directions focus on developing more robust architectures capable of handling increasingly complex, multi-domain workflows while maintaining operational transparency.

2.3 Taxonomy on Task Flow Management and Execution Mechanisms

The architectural design of task flow management [6, 21, 191] in AI-driven automation systems reflects fundamental trade-offs between predictability, efficiency, and adaptability. As detailed in Table 3, contemporary tools employ four distinct execution paradigms - sequential, parallel, iterative, and task decomposition-based approaches [31, 86, 192] - each demonstrating unique technical characteristics shaped by their underlying coordination models and error recovery strategies [39, 95, 99].

As shown in Figure 7 (1), *Sequential Execution Automation System* [14, 66, 69, 75] adopts a linear task execution model, where each task proceeds only after the previous one completes. This approach ensures simplicity, predictability, and ease of debugging, making it well-suited for workflows with fixed logic and minimal variability, such as form approvals or structured back-office operations [15, 34]. However, the strict step-by-step coordination limits scalability and responsiveness, making the system less adaptable to dynamic environments or exception-heavy scenarios [33, 177].

As illustrated in Figure 7 (2), *Parallel Execution Automation System* enables concurrent processing of multiple tasks using a graph-based dependency resolution framework [65, 130, 179]. By distributing workload across agents and allowing simultaneous execution, it significantly improves throughput and performance in multi-tasking or data-intensive applications [49, 50]. Nonetheless, the architecture introduces challenges in synchronization, consistency maintenance, and error tracking, which require robust monitoring and coordination mechanisms [184, 185].

As depicted in Figure 7 (3), *Iterative Execution Automation System* focuses on adaptive task flows by incorporating feedback loops and incremental workflow optimization [78, 96, 142]. This model is highly effective in environments where requirements evolve over time, such as conversational agents or adaptive planning systems, as it allows ongoing refinement of tasks based on observed outcomes. However, its complexity arises from the need for rollback mechanisms, state versioning, and audit trails to ensure

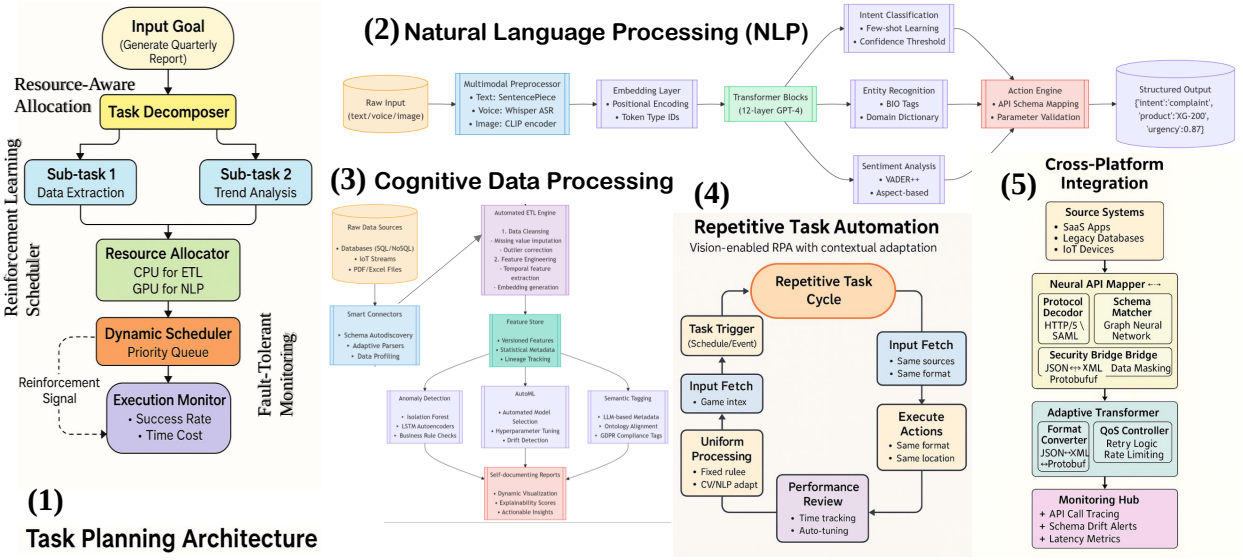


Figure 5: Representative Architectures of AI-Driven Automation Systems Across Functional Categories.

compliance and traceability during iterative updates [92, 94].

As shown in Figure 7 (4), *Task Decomposition Automation System* leverages goal-oriented reasoning to break down complex objectives into smaller executable sub-tasks [71, 72, 198]. By combining neural and symbolic techniques, it supports scalable task planning across diverse domains and agent specializations [31, 86]. This architecture excels in modularity and reuse, making it ideal for open-ended or cross-domain problems, though it demands sophisticated control strategies to manage coordination and task interdependencies effectively [39, 95].

2.3.1 In-depth Analysis of Each Execution Mechanisms

Sequential Task Execution: [66, 69, 75] Exemplified by UiPath [104, 106, 107] and Zapier [30], sequential models enforce strict linear execution through deterministic scheduling algorithms and finite-state machines. These systems prioritize operational reliability over flexibility, utilizing static priority queues and rule-based triggers to maintain process integrity. Their architecture proves particularly effective in legacy system integration and routine office workflows (e.g., employee onboarding approvals), where predictable patterns dominate. However, this rigid temporal coordination inherently limits throughput and dynamic exception handling, as task latency accumulates linearly and environmental changes require manual workflow redesign [33, 177, 178].

Parallel Task Execution: [65, 130, 179] Systems like OWL [120] and MetaGPT [60] implement concurrent processing through distributed task queues with graph-based dependency resolution. These architectures employ lock-free data structures and probabilistic conflict resolution to coordinate simultaneous execution across multi-agent environments. The technical innovation lies in real-time resource monitoring systems that balance computational load across agents while maintaining data consistency through semantic versioning protocols.

Iterative Task Execution: [78, 96, 142] LangChain [111, 140, 153] and OpenManus [114, 117] exemplify feedback-driven architectures that combine versioned workflow states with reinforcement learning policies. These systems implement semantic differencing algorithms to identify optimization opportunities between iterations, enabling adaptive adjustment of execution paths. The technical challenge centers on designing efficient rollback mechanisms and audit trails that preserve compliance while supporting dynamic workflow

Functionality	Current Maturity Stage	Estimated Time to Plateau	Key Rationale
Repetitive Task Automation	Plateau of Productivity / Late Slope of Enlightenment	Now - 2 Years	Rule-based RPA is mature; AI-enhanced RPA (UiPath, Blue Prism) commercially deployed at scale for specific scenarios (invoice processing, data migration) with standardized tech.
Cross-Platform Integration	Slope of Enlightenment	2 - 4 Years	Tools (Zapier, UiPath Connect) widely used but neural API mapping, adaptive protocol translation, and complex hybrid cloud/security integration still evolving.
NLP Integration	Slope of Enlightenment	3 - 5 Years	Core LLMs (GPT-4) powerful but workflow integration (context-aware parsing, multimodal instruction, adaptive dialog) needs reliability/control improvements. Application patterns still emerging.
Cognitive Data Processing	Late Peak / Early Trough	5 - 7 Years	Advanced concepts (auto feature engineering, cognitive ETL) with experimental systems (MetaGPT, OWL). Challenges in scaling and explaining automated semantic understanding.
Task Planning	Innovation Trigger / Peak of Inflated Expectations	7+ Years	Novel architectures (GNNs, hierarchical decomposition) in research frameworks (Manus, LangChain). Robustness and scalability for complex workflows need validation.

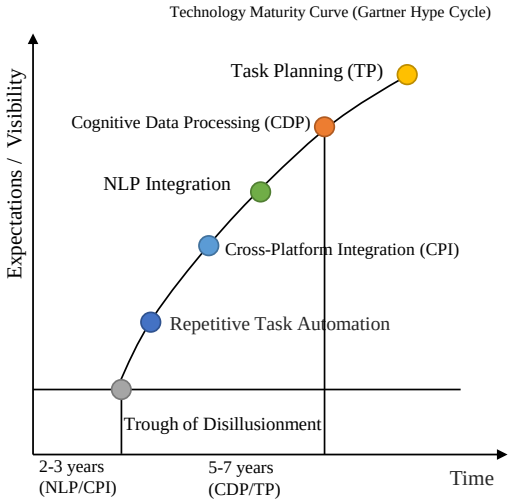


Figure 6: Technology Maturity Assessment of AI-Driven Automation Functional Categories.

evolution.

Task Decomposition Strategies: [71, 72, 198] Advanced frameworks like Manus [117, 119] and AutoGPT [113, 159] employ hybrid neuro-symbolic reasoning to decompose abstract goals into executable sub-tasks. Through multi-resolution analysis modules, these systems alternate between symbolic planning and neural subtask generation, enabling collaborative execution across specialized agents [31, 86]. The architectural divergence manifests in centralized versus distributed control philosophies - AutoGPT utilizes monolithic decomposition controllers, while OWL [120] implements decentralized negotiation protocols.

2.3.2 Comparative Analysis and Selection Guidelines

The choice of task execution paradigm requires careful consideration of operational requirements and system constraints [39, 95]. As shown in Table 4, each mechanism presents distinct trade-offs across five critical dimensions: implementation complexity, throughput efficiency, dynamic adaptability, modularity, and observability [184, 185].

Sequential execution offers minimal implementation complexity and straightforward debugging [15, 34], making it ideal for small-scale workflows with fixed logic patterns, though it sacrifices throughput and adaptability. Parallel architectures [49, 50] significantly enhance processing capacity for batch operations through concurrent resource utilization, but introduce synchronization challenges and increase monitoring overhead [186]. Iterative models excel in dynamic environments requiring continuous optimization, employing feedback loops to adapt workflows, albeit with elevated implementation and maintenance costs [92, 94]. Task decomposition strategies provide superior modularity for complex multi-domain tasks through specialized agent collaboration [115, 192], though demanding sophisticated coordination frameworks.

In conclusion, the evolution of these mechanisms reveals an industry-wide shift from rigid workflow automation to adaptive process orchestration [39, 71, 95]. Modern automation systems increasingly combine multiple paradigms, while facing persistent challenges in maintaining operational coherence across hybrid coordination models [65, 86, 130]. For instance, employing decomposition for task planning follows by parallel execution [31, 50, 49]. In future research, greater emphasis can be placed on self-organizing architectures capable of dynamically selecting execution strategies based on real-time system states and environmental constraints [92, 94, 187].

Table 3: Task Flow Management Architectures

Management Type	Technical Characteristics	Representative Systems
Sequential Execution	Linear state transitions with static dependency resolution	UiPath [104, 106], Zapier [30], AirSlate [122], HeyTAP [108], RecRules [110], ChatIoT [109], SmartFlow [55], LMRPA [33], AutoAgent [123]
Parallel Execution	Concurrent processing with graph-based dependency tracking	OWL [120], MetaGPT [60], AutoGen [91], AgentVerse [65], CrewAI [121], TradingGPT [16], GameGPT [48], MAgIC [193], LLMArena [134], ChatDev [47]
Iterative Execution	Feedback-aware workflow versioning with rollback mechanisms	LangChain [111, 112], OpenManus [114], TaskMatrix.AI [23], LLM-Planner [155], Reflexion [83], ToRA [154], ReAct [78], Self-RAG [139], Tree-of-Thought [96], AlphaCodium [194], LoopGPT [195]
Task Decomposition	Hybrid neuro-symbolic goal reduction architectures	Manus [117, 118], OWL [120], AutoGPT [113], ToRA [154], LLM-Planner [155], AgentVerse [65], ToolLLM [59], Mixtral of Experts [131], ExpeL [196], EconAgent [32], TidyBot [197]

Table 4: Comparative Analysis of Task Execution Paradigms

Characteristic Dimension	Sequential	Parallel	Iterative	Decomposition
Implementation Complexity	Low	Medium-High	High	Medium-High
Throughput Performance	Low	High	Medium	Medium-High
Dynamic Adaptability	Low	Medium	High	High
Modularity/Reusability	Low	Medium	Medium	High
Debugging & Monitoring Difficulty	Low	High	High	Medium-High

2.4 Application Taxonomy of AI-Driven Automation

2.4.1 Primary Benefited Production Aspects of AI-Driven Automation

The application landscape of AI-driven automation tools spans multiple functional domains, each demonstrating unique technical implementations tailored to specific operational requirements [71, 86, 95]. As detailed in Table 5, these solutions exhibit specialized architectural configurations that address distinct challenges across enterprise workflows.

Customer Support Automation: Modern implementations leverage transformer-based architectures like GPT-4 [156, 89, 199] and specialized systems such as Cogito [200] to establish multi-modal communication channels. These systems combine natural language understanding with dialog management engines to process customer inquiries through hybrid symbolic-neural approaches. The technical differentiation emerges in their ability to handle context switching between structured FAQs and unstructured conversational flows while maintaining compliance with industry-specific communication protocols [78, 92, 96].

Data Analysis & Reporting [14, 34, 75]: Tools like OWL [120] and UiPath AI Fabric [106, 107] demonstrate advanced data fusion capabilities, integrating structured database queries with unstructured text analysis through neuro-symbolic architectures. LangChain [111] introduces a novel chaining mechanism that connects disparate data sources through semantic linking, while MetaGPT [60] employs graph neural networks to visualize complex data relationships. These systems differ fundamentally from traditional Business Intelligence (BI) tools [17, 32, 209, 210] through their dynamic hypothesis generation capabilities and automated insight validation frameworks.

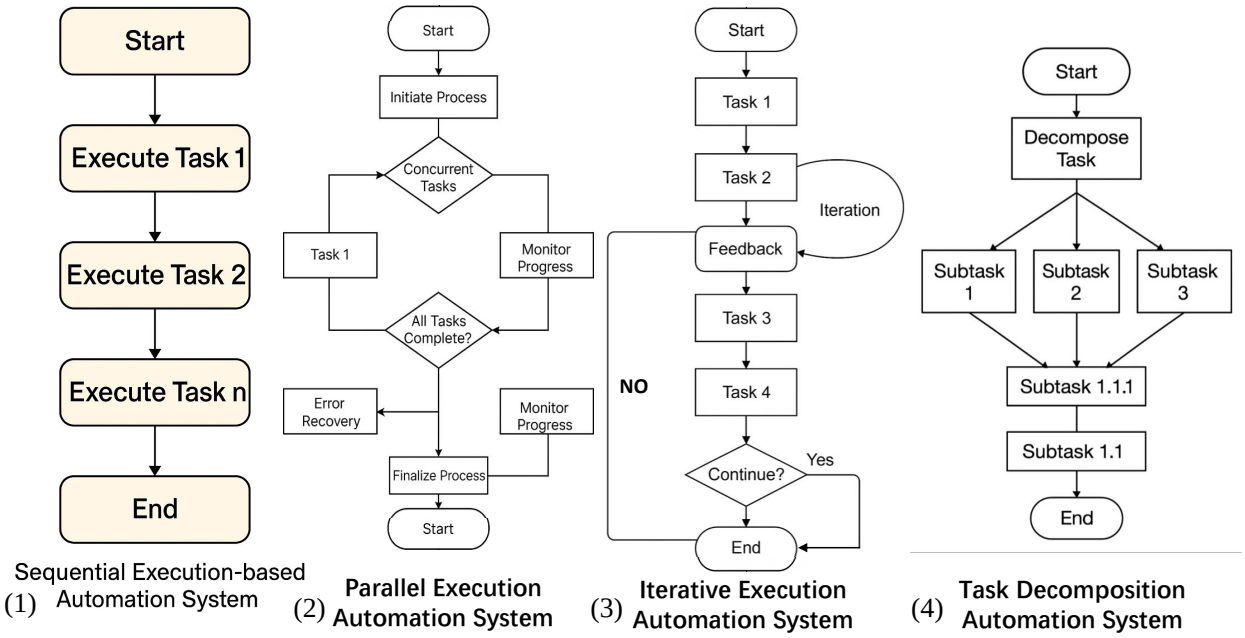


Figure 7: Representative Architectures of AI-Driven Automation Systems Across Functional Categories.

Document Processing & Content Generation [53, 72, 198]: This domain combines neural machine translation systems like DeepL [157, 207] with generative architectures in GPT-4 [45, 199] and structured document processors like Manus [117, 118, 119]. The technical innovation lies in multi-stage processing pipelines that first parse document semantics through attention mechanisms, then apply domain-specific templates, and finally generate context-aware outputs. LangChain's contribution emerges through its modular architecture that separates content generation from style formatting, enabling adaptive reuse of document components across different output formats [10, 112].

Operational Task Automation [33, 66, 177]: Systems such as UiPath [106, 107] and AirSlate [122] represent the evolution of traditional RPA [15, 66, 81] through computer vision-enhanced workflow recognition and self-adjusting execution engines. OpenManus [114] introduces a decentralized approach to operational automation through its agent-based task allocation system, particularly effective in distributed procurement processes. The technical differentiation manifests in their exception handling mechanisms, ranging from rule-based fallback systems in UiPath to neural pattern recognition in AirSlate's document-centric workflows [70, 178].

Cross-Platform Workflow Automation [65, 130, 179]: Zapier's [30] AI-enhanced integration framework and OWL's [120] multi-agent automation system exemplify two distinct technical approaches to platform interoperability [19, 31, 64, 192]. While Zapier [30] employs neural API mapping to bridge commercial SaaS applications, OWL [120] utilizes semantic protocol translation for industrial IoT environments. UiPath AI Fabric [106, 107] demonstrates a hybrid approach combining RPA connectors with machine learning-based workflow adaptation, particularly effective in legacy system integration scenarios. These systems share common challenges in maintaining data consistency across heterogeneous security models and authentication protocols [182, 183].

As illustrates in Figure 8, it presents the functional taxonomy and ecosystem architecture of prominent AI-driven automation tools. The tools are grouped into five distinct categories: *customer support automation, data analysis and reporting, document processing and content generation, operational task automation, and cross-platform workflow automation* [71, 86, 95]. Each category reflects a unique application domain and

Table 5: Application Domains of AI-Driven Automation Tools

Application Area	Technical Implementation	Representative Tools
Customer Support Automation	Transformer-based dialog systems with compliance-aware response generation.	GPT-4 [199, 201], Cogito [200], MetaGPT [60], Med-Bot [57], IntellBot [52], EduChat [202], NOVI [58], CyberMentor [203]
Data Analysis & Reporting	Neuro-symbolic data fusion architectures with automated insight validation.	OWL [120], MetaGPT [60], UiPath AI Fabric [106], CypherBench [124], Docling [53], S3 [204], RecMind [205], mAIstro [206]
Document Processing & Content Generation	Multi-stage neural pipelines with style-content decoupling.	DeepL [157, 207], GPT-4 [199], LangChain [111, 152], Docling [53], MiniRAG [138], Self-RAG [139], Rag-Fusion [208] , MM-REACT [79]
Operational Task Automation	Vision-enhanced RPA with adaptive exception handling.	UiPath [106, 107], OpenManus [114], AirSlate [122], SmartFlow [55], LMRPA [33], BreachSeek [163], AutoAgent [123], TidyBot [197]
Cross-Platform Workflow Automation	Neural API mapping with protocol translation engines.	Zapier [30], OWL [120], UiPath AI Fabric [106], ToolLLM [59], AutoGen [91], AgentVerse [65], CrewAI [121], TaskMatrix.AI [23]

technological foundation, ranging from traditional RPA systems to advanced multi-agent automation systems and cognitive orchestration engines [60, 65, 111].

The visual layout highlights the vertical stack of automation complexity, starting from foundational tools like UiPath and Zapier [30, 106] at the infrastructure layer [66, 75], progressing through mid-layer platforms such as LangChain and MetaGPT [60, 111] that facilitate reasoning and task decomposition, and culminating in agent-based ecosystems like Manus [117] and OWL [120] that enable decentralized coordination and emergent behavior [86, 92]. By mapping tools according to both functionality and architectural sophistication, the figure provides a comparative perspective on how modern AI systems address varying degrees of autonomy, interoperability, and domain specialization within automation workflows [71, 95].

Overall, Figure 8 explains the role of each tool in automation while revealing new trends like LLM-based reasoning working with multi-agent automation systems. It also points toward the future development of fully adaptive, cognitive automation stacks [94, 187].

In conclusion, the application-specific implementations reveal three fundamental technical trends: 1) Increasing specialization of neural architectures for domain-specific constraints [72, 198], 2) Growing emphasis on hybrid symbolic-connectionist systems for enterprise compliance requirements [78, 96], and 3) Emergence of multi-paradigm integration frameworks to handle heterogeneous automation environments [65, 130]. Researchers are currently tackling the challenge of enhancing cross-domain generalization capabilities, while preserving application-specific optimization [85, 111, 211, 212].

2.4.2 Industry-Specific Implementation and Production Scenarios

The convergence of industry-specific requirements and application-driven architectures has shaped the evolution of AI-driven automation technologies [14, 34]. As shown in Figure 9, we provide a structured classification of AI-driven automation architectures across five major industry / human living scenarios: *Manufacturing* [20, 81], *Healthcare* [57, 14, 206], *Finance* [16, 213], *Energy* [40, 167], and *Education* [202, 203]. Each column outlines the key applications, underlying technical frameworks, and representative systems currently deployed in practice. This layered visual categorization captures both the breadth and specificity of AI implementations across domains, ranging from predictive maintenance in manufacturing

Customer Support Automation

Tool	Main Features Description
Freshdesk	AI-powered customer service automation, including smart ticket routing, auto-reply suggestions, and omnichannel support.
Tidio	AI chatbot integration with live chat, email, and social media support — ideal for SMBs.
Zoho Desk	AI assistant *Zia* automates ticket categorization, prioritization, and response time prediction.
Zendesk	Multichannel support with AI automation to improve service efficiency.
Intercom	Personalized support via AI assistant; supports multichannel communication and automated workflows.

Data Analysis & Reporting Automation

Tool	Main Features Description
UiPath AI Fabric	Integrates ML models to automate data processing and report generation.
MetaGPT	Uses graph neural networks + NLP to generate analytical reports automatically.
LangChain	Chain-based framework to process multi-source data and create structured reports.
OWL	Supports multimodal data fusion and automated insight validation.
DeepL	High-quality machine translation, ideal for multilingual data processing.

Document Processing & Content Generation

Tool	Main Features Description
GPT-4	Versatile natural language generation for diverse text creation tasks.
LangChain	Modular structure supporting separation of content generation and formatting.
DeepL	Accurate multilingual translation, enabling global content workflows.
Manus	Structured document automation for scalable content generation.
OpenManus	Decentralized platform for co-editing and document generation.

Operational Task Automation

Tool	Main Features Description
UiPath	RPA-based visual task automation, enabling intelligent and efficient process execution.
AirSlate	Automates workflows and documents from end to end, with seamless multi-platform integration.
OpenManus	Decentralized task automation platform with smart task assignment and execution support.
Cogito	Provides sentiment analysis and conversation management to improve customer interaction quality.
MetaGPT	Multi-agent coordination engine for efficient cross-platform task automation and orchestration.



Cross-Platform Workflow Automation

Tool	Main Features Description
Zapier	Connects apps and automates workflows via API mapping and protocol conversion.
OWL	Multi-agent system with semantic protocol conversion for IoT environments.
UiPath AI Fabric	Combines RPA and AI to enable seamless cross-platform automation.
MetaGPT	Coordinates intelligent agents for distributed workflow execution.
ServiceNow	AI-powered platform for workflow automation and system integration.

Figure 8: Primary Functional Taxonomy and System Architecture of Contemporary AI-Driven Automation Tools.

to adaptive assessment in education [68, 202, 203, 214]. For instance, manufacturing leverages edge-cloud hybrid computing and 3D vision systems [25, 211] to support physical automation, while healthcare systems prioritize privacy-preserving AI and compliance frameworks [13, 215].

Meanwhile, in Table 6 presents a unified taxonomy that cross-references industrial domains with their characteristic applications, technical implementations, and representative systems [17, 35, 209].

The integrated analysis reveals several critical architectural patterns [65, 71, 86]. Manufacturing systems exemplify the tight coupling between physical automation and AI decision-making, where the mainstream generative AI engine coordinates with industrial robots through sub-millisecond control loops [25, 68, 211]. This contrasts with healthcare implementations that prioritize ethical constraints, as seen in *BD HemoSphere Alta* [218]'s differential privacy mechanisms that anonymize patient data while maintaining diagnostic accuracy [98, 215].

Financial sector tools demonstrate unique hybrid architectures, where AirSlate [122] combines NLP-driven document processing with regulatory compliance checks through neural-symbolic reasoning. The system dynamically generates audit trails by tracing data transformations across workflow steps, achieving both operational efficiency and compliance transparency. Similarly, education technologies like OneClickQuiz [221, 222] implement Bloom's Taxonomy through distilled language models, enabling rapid question generation while maintaining pedagogical alignment [202, 203].

Energy management solutions [40, 167, 216] highlight the growing importance of distributed intelligence, with OWL's [120] multi-agent automation system optimizing grid operations through edge-based predictive models [223, 224]. These architectures employ specialized hardware-software codesign to withstand harsh environmental conditions while maintaining real-time processing capabilities [19, 18]. Across all domains, the convergence of application-specific requirements and industrial constraints continues to drive architectural innovation in AI-driven automation systems [94, 95, 142].

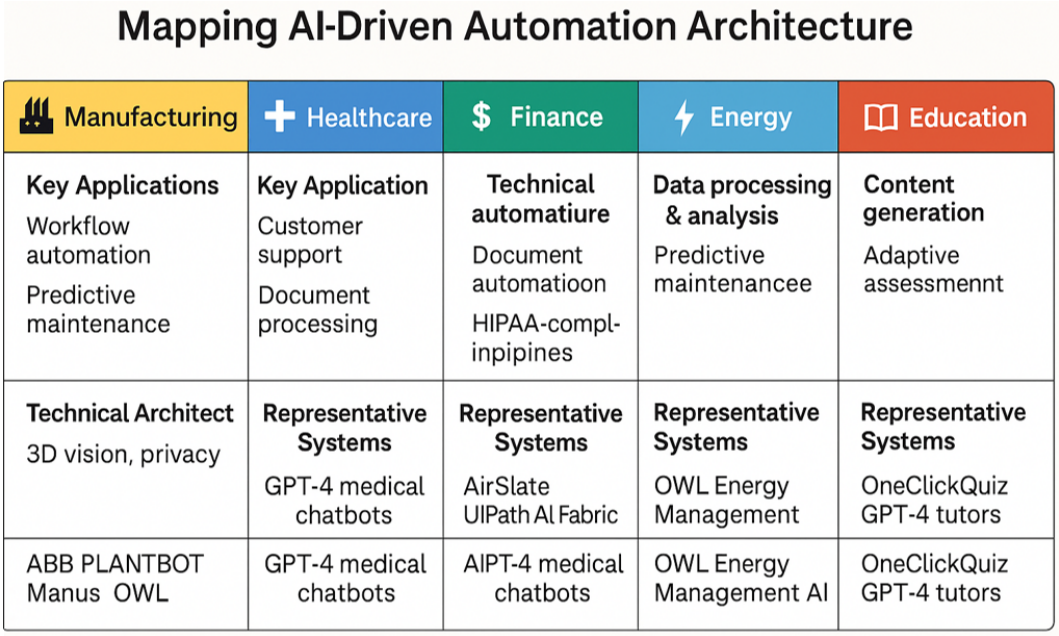


Figure 9: Primary Functional Taxonomy and System Architecture of Contemporary AI-Driven Automation Tools.

2.4.3 Lightweight AI-Driven Automation for Edge Devices, Mobile Systems, or Low-Resource Environments

The surge in Internet of Things (IoT) devices and the increasing deployment of mobile systems and embedded technologies have created a significant need for lightweight AI-driven automation solutions [223, 224]. As a result, traditional cloud-based AI solutions, which typically rely on robust computational infrastructures, are not suitable for deployment in these low-resource environments. This section discusses how lightweight AI-driven automation systems are addressing these challenges and enabling effective automation for edge devices, mobile systems, and other low-resource environments.

1. Edge Device Automation: Edge devices, including sensors, cameras, and gateways, play a pivotal role in modern automation by collecting and processing data locally to minimize latency and reduce dependence on cloud infrastructure [220]. These devices often operate in environments where bandwidth is limited or unreliable, making cloud communication less efficient. To address this, edge AI systems are designed with compact models that execute inference tasks with minimal resources. Techniques such as model quantization, pruning, and knowledge distillation are frequently employed to compress large, resource-intensive models into smaller, more efficient versions without sacrificing accuracy [88, 89]. For instance, lightweight convolutional neural networks (CNNs) and decision tree-based models have gained popularity for tasks like object detection, and predictive maintenance on embedded systems [21].

2. Mobile Systems and IoT Automation: Mobile systems and IoT devices often have limited power budgets and processing capabilities, yet they need to perform tasks like speech recognition, image processing, and sensor data analysis [160, 162]. To enable AI automation on such devices, specialized AI chips and microcontrollers are integrated to offload computationally heavy tasks from the main processor, thus optimizing power usage [223]. This empowers edge devices and mobile systems to perform tasks like image recognition, natural language processing, and personalized recommendations locally [225].

3. Embedded Systems in Low-Resource Environments: Embedded systems in domains like agriculture, healthcare, and industrial automation are increasingly adopting lightweight AI-driven automation solutions [68, 211, 214]. These systems often rely on custom-designed hardware to meet performance and power constraints, which enable efficient AI processing with minimal power usage [224]. In these contexts,

Table 6: Integrated Taxonomy of AI-Driven Automation by Industry and Application

Industry	Key Applications	Technical Architecture	Representative Systems
Manufacturing	Complex workflow management, Predictive maintenance	Multi-modal perception (3D vision, force sensing), Real-time adaptive control, Edge-cloud hybrid computing	GPT Researcher [44], BOLAA [56], TPTU-v2 [68], Manus [117, 119], AutoGPT [113], SuperAGI [116], OWL [120], ABB PLANTBOT [216], WorkGPT [217]
Healthcare	Customer support automation, Document processing	High-precision sensor fusion, Privacy-preserving AI, HIPAA-compliant data pipelines	MedAgents [12], GPT-4 medical chatbots [13, 105], Med-Bot [57], mAIstro [206], D-Bot [215], *BD HemoSphere Alta* [218], LLM-TA [219]
Finance	Document automation, Cross-platform integration	Neural-symbolic compliance engines, Dynamic audit trails, SOX/GDPR embedded architectures	TradingGPT [16], Financial RPA [17], EconAgent [32], AirSlate [122], UiPath AI Fabric [106, 107], CompeteAI [210], SmartFlow [55]
Energy	Data processing & analysis, Predictive maintenance	Distributed edge AI, Multi-agent grid optimization, Ruggedized hardware integration	OWL Energy Management [120], DeChen Edge AI [220], LLM-OptiRA [167], Hyperautomation [27], Intelligent Automation [211]
Education	Content generation, Adaptive assessment	Cognitive framework alignment, Multi-modal interaction, Pedagogical model integration	NOVI [58], MiniRAG [138], EduChat [202], GPT-4 tutors [199, 201], CyberMentor [203], OneClickQuiz [221, 222]

techniques like model compression, low-precision arithmetic, and hardware acceleration help make the automation feasible in low-resource environments [21].

4. The Future of Lightweight AI-Driven Automation: Looking ahead, several trends will further enhance the deployment of AI-driven automation on edge and low-resource devices. Federated learning, for instance, allows devices to collaboratively learn from data without transmitting sensitive information to the cloud, making it ideal for environments where privacy is paramount [223]. Furthermore, advances in AI chip design, including the integration of AI accelerators into microcontrollers, will continue to push the boundaries of what is possible in edge and mobile AI [224].

In conclusion, lightweight AI-driven automation is a rapidly evolving field that is crucial for enabling intelligent systems in edge devices, mobile systems, and low-resource environments [223, 224]. These advancements will help bridge the gap between cloud-based automation and real-time, on-device processing, bringing AI-powered automation to a wide range of applications with minimal resource requirements.

2.5 Summary

This section systematically established a novel taxonomy for classifying AI-driven automation technologies, addressing the critical need for a unified framework in this rapidly evolving field. Our analysis delineates four foundational architectural paradigms:

1. Rule-Based Systems: Characterized by deterministic workflows, suitable for structured environments but limited in adaptability.

2. LLM-Driven Approaches: Leveraging natural language understanding for dynamic task orchestration, though challenged by interpretability and data dependence.

3. Multi-Agent Cooperative Models: Enabling complex, distributed problem-solving through agent collaboration, with trade-offs in coordination overhead.

4. Hybrid Architectures: Combining strengths of the above paradigms to balance flexibility and

robustness, as exemplified by tools like Manus [117] and OWL [120].

Further, we categorize these technologies by functionality (e.g., NLP-centric vs. RPA-integrated), execution mechanisms (sequential vs. parallel), and human-AI collaboration modes (assistive vs. autonomous). This not only clarifies the current landscape but also highlights gaps in interoperability and scalability, which is the key challenges illustrated in Section 4.

3 Recent Mainstream Applications of AI-Driven Automation

3.1 Overview of Prominent AI-Driven Automation Tools

Recent years have witnessed the emergence of diverse AI-driven automation tools, each addressing distinct challenges across industries. The evolution of AI-driven automation has entered a transformative phase characterized by three fundamental shifts [6, 71, 95]: (1) from static rule-based systems to dynamic cognitive architectures [78, 96, 97], (2) from single-agent operations to collaborative multi-agent ecosystems[16, 31, 65, 115], and (3) from domain-specific solutions to cross-platform cognitive frameworks [9, 59, 111].

As detailed in Table 7, we list several major representative AI-driven automation tools, accompanied by concise descriptions of their core functionalities and technological underpinnings. This progression manifests through distinct technological generations that address emerging challenges in social production automation [25, 27, 211].

3.1.1 Architectural Paradigm Shifts

The evolution of AI-driven automation tools demonstrates a clear architectural progression from rule-based systems to cognitive architectures [75, 77, 105]. Early hybrid RPA-AI solutions like UiPath AI Fabric [106, 107] have given way to sophisticated neuro-symbolic systems exemplified by post-2023 developments such as LangChain [10, 111, 152]. These modern frameworks introduce cognitive chaining capabilities, where LLMs dynamically orchestrate sequential reasoning processes through API composition [59, 128]. The transition reflects a broader industry movement from deterministic automation to adaptive, knowledge-driven workflow systems that demonstrate contextual awareness and learning capabilities.

3.1.2 Emerging Design Patterns

Analysis of development methodologies reveals two significant innovation patterns in the field. The academic-industrial fusion model, exemplified by MetaGPT [60]'s joint development between DeepWisdom and KAUST [227], successfully combines industrial-grade scalability [68, 211] with academic research rigor in multi-agent coordination.

Concurrently, the open-source ecosystem approach demonstrates remarkable efficacy, as evidenced by LangChain's rapid community adoption, showcasing how community-driven development can accelerate toolchain maturation [111, 228]. These AI-driven automation tools collectively establish a new *cognitive automation stack* comprising four layers: (1) Sensorimotor interfaces [4, 108, 229], (2) Neural reasoning cores [5, 141, 142], (3) Multi-agent coordination planes [127, 129], and (4) Evolutionary adaptation mechanisms [39].

Overall, these above introduced tools exemplify the convergence of AI methodologies, such as LLMs, multi-agent automation systems, and hybrid rule-AI architectures, to address scalability, adaptability, and interoperability challenges in modern automation applications.

3.2 Detailed Discussion of Several Representative AI-Driven Automation Tools

3.2.1 Manus (From Butterfly Effect)

As a representative implementation of a multi-agent automation system, Manus [117, 119] operates within an isolated virtual environment and is composed of three primary functional modules: the Planning

Table 7: Latest Statistics of Several Prominent AI-Driven Automation Tools

AI Automatic Tools	Release Date	Development Team	Characteristic and Capability Description
GPT-4 Series [199, 201]	March 2023	OpenAI	Versatile NLP models integrated via APIs to automate content generation, conversational interfaces, and semantic analysis in broader workflows.
LangChain [112, 111]	October 2022	Harrison Chase	An open-source framework for building language model-powered applications, enabling chained task execution, text generation, and seamless integration with external APIs.
Manus [117, 119]	March 2025	Butterfly Effect (Monica.im)	A multi-agent automation system with autonomous task planning, execution tracking, and adaptive memory modules for iterative workflow optimization.
OpenManus [114]	March 2025	MetaGPT	A modular extension of Manus, designed for customizable agent deployment in domain-specific automation scenarios.
OWL [120]	March 2025	CAMEL-AI	A scalable, open-source multi-agent automation system supporting cross-platform operations and heterogeneous data source integration for complex industrial workflows.
AutoGPT [113]	April 2023	Toran Bruce Richards	A self-directed AI agent combining GPT architectures with reinforcement learning to autonomously generate and execute task sequences.
MetaGPT [60]	July 2023	DeepWisdom & KAUST	A multi-agent automation system optimized for multimodal task allocation, coordination, and dynamic resource management.
UiPath AI Fabric [106, 107]	October 2020	UiPath	Enhances robotic process automation (RPA) with AI-driven decision-making, enabling hybrid rule-based and machine learning workflows for enterprise tasks.
Zapier [30]	May 2023	Zapier Inc.	A low-code platform enabling cross-application workflow automation through AI-augmented rule engines and API orchestration.
AirSlate [122]	December 2020	AirSlate Inc.	Combines RPA with AI for automating document-centric processes such as contract management and form processing.
AI-enhanced Tray.io [226]	February 2024	Tray.io Inc.	An enterprise-grade integration platform utilizing AI to streamline API-driven workflow automation across distributed systems.

Module, the Memory Module, and the Tool Use Module. The left part of Figure 10 illustrates the core architectural design of the *Manus* multi-agent automation system. These components work collaboratively to enable Manus to interpret, plan, and execute complex tasks in an automated and adaptive manner.

The Planning Module serves as the "brain" of the system, responsible for understanding user intentions, decomposing abstract tasks into concrete actions, and generating executable plans. It supports semantic understanding (NLU) [78, 96, 173], priority sorting [71, 86], resource allocation [25, 167], DAG-based task decomposition [142, 230], and exception handling. The Memory Module empowers the system to maintain coherence and personalization across sessions by storing user preferences, interaction history, and intermediate results. A typical implementation may include components such as user profiling vectors, interaction history databases (e.g., ChromaDB [231]), and short-term memory caches [83, 232]. Finally, the Tool Use Module acts as the system's "hands", integrating capabilities such as web search, data processing, code execution [130, 233, 234], and content generation. This multi-tool orchestration allows Manus to effectively complete diverse and sophisticated tasks.

As illustrated on the right side of Figure 10, Manus operates based on a structured task execution flow under a *Multiple Agent Architecture* [65, 92, 94]. The system runs within an independent virtual environment and is capable of handling both simple queries and complex project-level requests. Upon receiving a task input from the user, the system initiates a multi-stage process to understand, decompose, and complete the

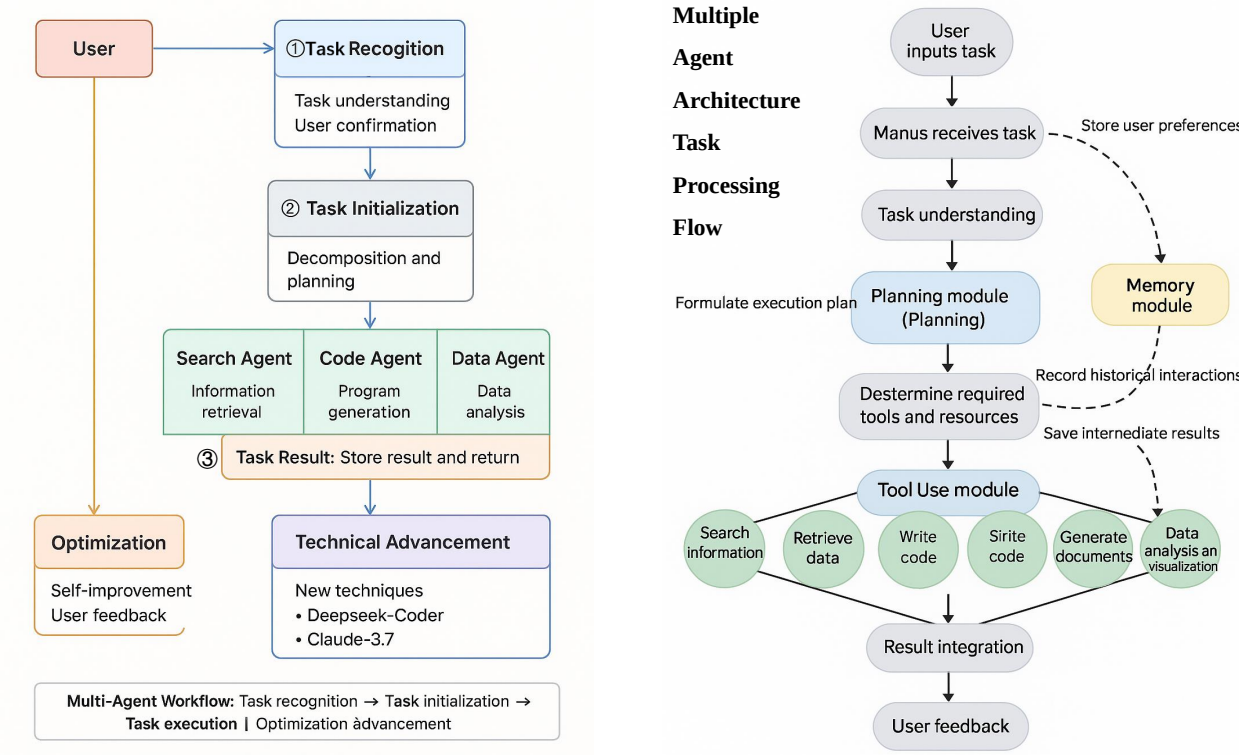


Figure 10: (1) Left: The Core Module Design and Composition of the Manus Multi-Agent Automation System; (2) Right: Manus Task Processing Flow under Multiple Agent Architecture.

task effectively.

The first step in this pipeline is **task reception**, where Manus accepts user input in various formats such as text, images, and documents [79, 235]. This is followed by the **task understanding** phase, in which Manus leverages NLP techniques to extract key intents and semantic elements from the input [78, 96]. During this phase, the Memory Module provides contextual support by supplying previously stored user preferences and interaction history, thereby enhancing personalization and interpretability [64, 67, 83]. When user goals are ambiguous, the system engages in interactive dialogue to help refine task objectives [92, 187].

Once the task intent is clarified, the **task decomposition** stage begins. Here, the Planning Module breaks down complex tasks into a series of executable subtasks, establishes dependencies, and determines execution order [142, 230]. This decomposition process typically results in a directed acyclic graph (DAG) [71, 198] structure that organizes task execution in a logical and efficient manner. Through this layered and modular architecture, Manus achieves robust adaptability and automation in diverse real-world applications [86, 114, 117].

As shown in Table 8, the architecture of Manus is organized into six distinct layers, each responsible for a critical aspect of task execution within the multi-agent automation system. The user interface layer enables interaction through a command line or REST API [23, 59, 128, 217], serving as the system's entry point. The control layer, powered by the flow engine, orchestrates task requests and initiates execution processes. At the core layer, planning and invocation flows are designed and instantiated to coordinate agent operations. The agent layer includes specialized agents-planning, execution, and validation-that work collaboratively to decompose tasks, execute actions, and verify outcomes. The tool layer provides essential capabilities such as web search, code execution, and result validation. Finally, the infrastructure layer ensures system stability and extensibility through configuration management, logging, tool registration, and reusable task templates.

Table 8: Layered Architecture of the **Manus**'s Task Execution System [117]

Layer	Component	Function
User Interface Layer	Command Line Interface / REST API	Users can send instructions or requests via the CLI or REST API to enter the system's flow engine for processing.
Control Layer	Flow Engine	As the core control center, it receives user requests, parses them, and forwards the demand to the planning engine, invoking agents and creating execution flow instances.
Core Layer	Planning Flow / Invocation Flow / Flow Instance	Planning Flow: Responsible for execution plan design. Invocation Flow: Responsible for execution path and agent invocation. Flow Instance: Executes specific instance flows, connecting and coordinating agents to perform tasks.
Agent Layer	Planning Agent / Execution Agent / Validation Agent	Planning Agent: Responsible for task decomposition, tool invocation, and task orchestration. Execution Agent: Executes specific tasks (e.g., running code). Validation Agent: Validates whether task results meet expectations.
Tool Layer	Search Tool / Code Tool / Validation Tool	Search Tool: Provides information retrieval capabilities. Code Tool: Executes scripts or performs code generation. Validation Tool: Verifies if task results are valid.
Infrastructure Layer	Configuration System / Logging System / Tool Registration Center / Template Repository	Configuration System: Provides configuration parameters to support planning agents. Logging System: Records runtime logs for agent auditing and traceability. Tool Registration Center: Manages tools and supports agent execution. Template Repository: Stores reusable task templates for planning agents.

Technical Features of Manus: Manus [117] stands out in the field of AI agents due to its advanced architecture and a range of technical innovations that enable highly autonomous and efficient task execution. One of its most distinguishing features is its **autonomous planning capability** [114, 118]. Unlike conventional tools that rely heavily on user input or predefined workflows, Manus can independently reason, plan, and execute tasks. In the General AI Assistant Benchmark (GAIA) [236], which evaluates real-world problem-solving capabilities of general-purpose AI assistants, Manus [117, 119] achieved state-of-the-art (SOTA) performance with a 94% success rate in completing complex tasks automatically.

Another key strength of Manus lies in its **context understanding** [117]. The system excels at interpreting vague or abstract user inputs, allowing it to identify user needs with minimal instruction. For instance, a user can describe the content of a video, and Manus will locate the corresponding link across platforms. This context-awareness supports sustained multi-turn interactions [83, 92, 187], with the system capable of maintaining coherent dialogues over ten or more turns, greatly enhancing the user experience.

Manus is built upon a **multi-agent architecture**, enabling collaborative execution of tasks across different functional agents. Similar to Anthropic's Computer Use paradigm [117, 119], Manus runs within a secure and isolated virtual machine, allowing specialized agents to handle planning, memory, and tool usage in parallel. This modular design ensures both scalability and fault tolerance when dealing with complex workflows.

Furthermore, **tool integration** is a hallmark of Manus's operational strength [59, 123, 237]. It can

automatically invoke and coordinate various tools for web search, data analysis, code generation, document creation, and more. This flexible integration allows Manus to handle a wide spectrum of tasks, from information retrieval to sophisticated analytical and creative work. The architecture also supports custom tool plugin development, making it extensible to specific application domains [65, 94, 114].

Finally, Manus emphasizes **security and system robustness**. By executing tasks within a gVisor-based sandboxed environment [51, 134, 223, 224], the system ensures process isolation, resource control, and execution safety. Combined with intelligent task scheduling mechanisms and modular agent design, Manus maximizes both efficiency and adaptability while maintaining high standards of stability and security [114, 118].

3.2.2 OpenManus (From MetaGPT Team)

The innovative design of OpenManus [114] centers around building a highly lightweight agent framework that emphasizes modularity and extensibility. It defines the functionality and behavior of agents through a combination of pluggable tools and prompts, significantly lowering the barrier for developing and customizing agents [21, 94, 147]. Prompts determine the agent's reasoning logic and behavioral patterns, while tools provide actionable capabilities such as system operations, code execution, and information retrieval. By freely combining different prompts and tools, new agents can be rapidly assembled and empowered to handle a wide range of task types [31, 113, 115, 238].

As depicted in Figure 11, the OpenManus Automation Framework consists of four key components: *a modular multi-agent automation system, tool integration, configuration and model support, and a bottom-up execution workflow*. At its core, the framework includes specialized agents such as the Manus Main Agent [117, 119], Planning Agent [148, 175], ToolCall Agent [59, 123], and customizable xAgents [92, 94], each responsible for planning, task execution, and tool invocation. These agents interact seamlessly with integrated tools like vector databases [144, 150], Python/NLP libraries [158, 157], web APIs [59, 237], and data visualization modules. The framework is further supported by flexible configuration files and multi-model integration capabilities, while the execution process is managed through a MetaGPT-based workflow [60, 227] and a dynamic task queue system [8, 56], ensuring scalable and efficient agent collaboration.

Essentially, OpenManus [114] is a multi-agent automation system. Unlike the one-shot [46, 72], all-in-one response style of a single large model, a multi-agent automation system tackles complex real-world problems through an iterative cycle of planning, execution, and feedback. In the design of OpenManus, the core concept is illustrated on the right side of Figure 11.

3.2.3 Optimized Workforce Learning for General Multi-Agent Assistance (OWL)

Optimized Workforce Learning for General Multi-Agent Assistance (OWL) [120] from Camel-AI[8] is a cutting-edge framework for multi-agent collaboration that pushes the boundaries of task automation, built on top of the CAMEL-AI Framework[9]. The left side of Figure 12 shows the system architecture overview of OWL. As seen in this sub-figure, the architecture of OWL consists of three main components: User Query, Actor Agents, and Tools Pool [31, 113, 120]. The system starts with a User Query, where the user initiates a task request. Actor Agents form the core module responsible for intelligent coordination, while the Tools Pool offers external interfaces needed for operations like web browsing, document parsing, and code execution [59, 237].

Among the Actor Agents, the AI User Agent and Assistant Agent work together to manage task decomposition and coordination [31, 113, 115]. The AI User Agent acts as the task initiator, and the Assistant Agent handles scheduling. Supporting them are functional agents-such as Web, Search, Coding,

[8]Camel-AI: https://www.camel-ai.org/.
[9]OWL Github: https://github.com/camel-ai/owl.

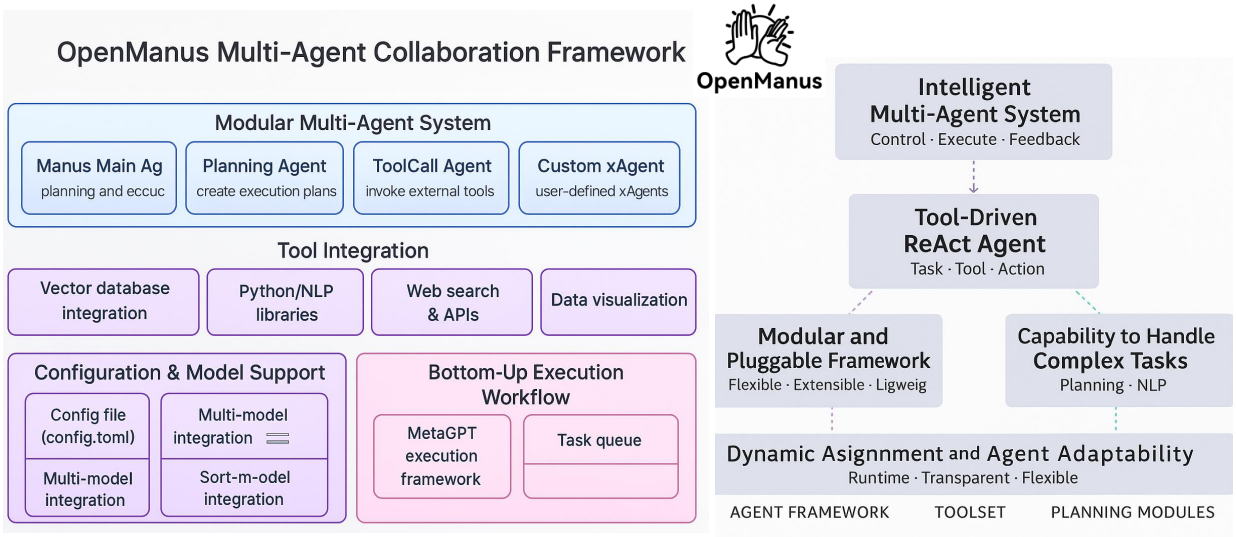


Figure 11: (1) Left: The roles and relationships among the agents in OpenManus; (2) Right: The architectural design philosophy of OpenManus.

and Document Agents-each responsible for specific tasks like browser interaction, information retrieval, code execution, and document parsing [59, 237].

The core functionalities of OWL include real-time information retrieval through online search engines such as *Wikipedia* [144] and *Google Search* [150], as well as multimodal processing capabilities that support videos, images, and audio from both internet sources and local storage. It enables automated browser interactions using the *Playwright* framework, allowing for actions like scrolling, clicking, typing, downloading, and navigating through web pages. Additionally, OWL supports document parsing, extracting information from Word, Excel, PDF, and PowerPoint files and converting the content into plain text or Markdown [53, 229]. It also features code execution capabilities, allowing users to write and run Python code through an integrated interpreter.

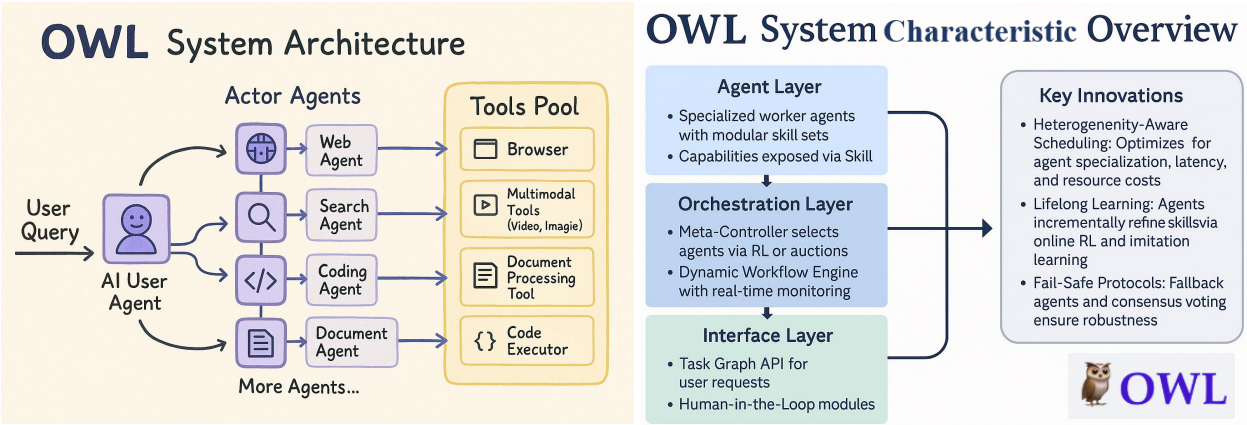


Figure 12: (1) Left: The system architecture overview of OWL; (2) Right: The system technical implementation overview of OWL.

The optimized workforce Learning for general multi-agent assistance (OWL) [120] framework is designed as a scalable, adaptive, and decentralized system for coordinating heterogeneous AI agents in

dynamic environments. The right side of Figure 12 shows a comprehensive feature and technical description of the OWL system. Its architecture consists of four interconnected layers: Agent Layer, Orchestration Layer, Learning Layer, and Interface Layer.

Specifically, (1). *Agent Layer* hosts a diverse pool of specialized worker agents (e.g., LLM-based, rule-based, or hybrid), each with modular skill sets. These agents expose their capabilities through a standardized `Skill API`, enabling dynamic task delegation and interoperability [31, 113, 115, 238]. (2). *Orchestration Layer* employs a `Meta-Controller` to optimize agent selection via reinforcement learning (RL) [85, 168] or auction-based mechanisms [16, 65], ensuring efficiency, cost-effectiveness, and constraint satisfaction. Additionally, a `Dynamic Workflow Engine` constructs multi-agent automation pipelines (modeled as DAGs) for complex tasks, with real-time performance monitoring and adaptation [8, 56]. (3). *Learning Layer* integrates `Federated Learning` to enable secure knowledge sharing among agents without raw data exchange, preserving privacy. It also leverages `Cross-Agent Transfer Learning` to generalize skills across domains, using shared embedding spaces or policy distillation techniques [135, 175]. (4). *Interface Layer* provides a `Unified Task Graph API` to abstract high-level user requests into decomposable subtasks [31, 113, 115]. It also incorporates `Human-in-the-Loop (HITL)` modules [188, 189], offering explainability dashboards and corrective feedback channels for human oversight.

Overall, OWL [120]'s architecture is agnostic to agent paradigms (LLMs, robots, etc.), making it suitable for enterprise automation, embodied AI, and large-scale assistive systems. With its flexible modular design and powerful tool integration capabilities, OWL is becoming a key foundation for open-source AI-driven automation applications.

3.2.4 LangChain (From Harrison Chase @LangChain Inc.)

LangChain [111, 112] is a modular framework designed to facilitate the development of multi-language and multi-scenario AI applications. As a widely adopted open-source framework, LangChain is designed to assist developers in building artificial intelligence applications. By providing standardized interfaces for chains, agents, and memory modules, LangChain simplifies the development process of applications based on LLMs.

Specifically, as illustrated in Figure 13, the architecture adopts a layered design that promotes extensibility, modularity, and cross-platform compatibility. Each layer in the stack is responsible for a specific aspect of system functionality, from foundational APIs [80, 128] to high-level cognitive orchestration [74].

At the top of Figure 13, the **LangServe** module provides the capability to deploy LangChain [111] workflows as RESTful APIs [239]. This makes it easy to integrate AI-powered chains into production environments, particularly within microservice architectures. Built with Python[10], LangServe offers a standardized interface for external systems to call AI logic seamlessly. Beneath the deployment layer, LangChain offers a set of *Templates* and *Committee Architectures*, which serve as reference implementations for common tasks like question answering and document analysis. The heart of the system lies in the **LangChain** module itself [111]. This layer defines three primary capabilities: *Chains* for sequential task composition, *Agents* for dynamic decision-making, and *Retrieval Strategies* for data fetching based on similarity or keywords.

As shown in the center of Figure 13, the **LangChain-Community** layer includes a rich ecosystem of pluggable modules for Model I/O, Prompting, Output Parsing, Document Loading, Vector Store integration, and Agent Toolkits. This layer enables developers to rapidly prototype and extend LangChain-based applications using components such as FAISS [240], Milvus [241], and external LLM APIs (e.g., ChatGPT [156, 45, 199] or LlaMa [88, 89]).

At the bottom of Figure 13, the **LangChain-Core** module defines abstract interfaces and reusable

[10]Python.Org: https://www.python.org/.

infrastructure for chains and agents. Alongside it, the *LangChain Expression Language (LCEL)* [242] provides a declarative syntax for chaining components. LCEL significantly reduces boilerplate code by allowing developers to describe "what to do" instead of "how to do it", improving focus on business logic while supporting streaming, async, and service composition.

LangChain's architecture supports a wide range of use cases, from retrieval-augmented generation (RAG) [138, 139, 208, 229] to multi-agent collaboration workflows [126, 175]. With built-in support for both Python and JavaScript[11], LangChain is suitable for backend, frontend, and hybrid AI systems. Its modularity allows for rapid iteration, and its deployment tooling ensures a smooth transition from prototyping to production.

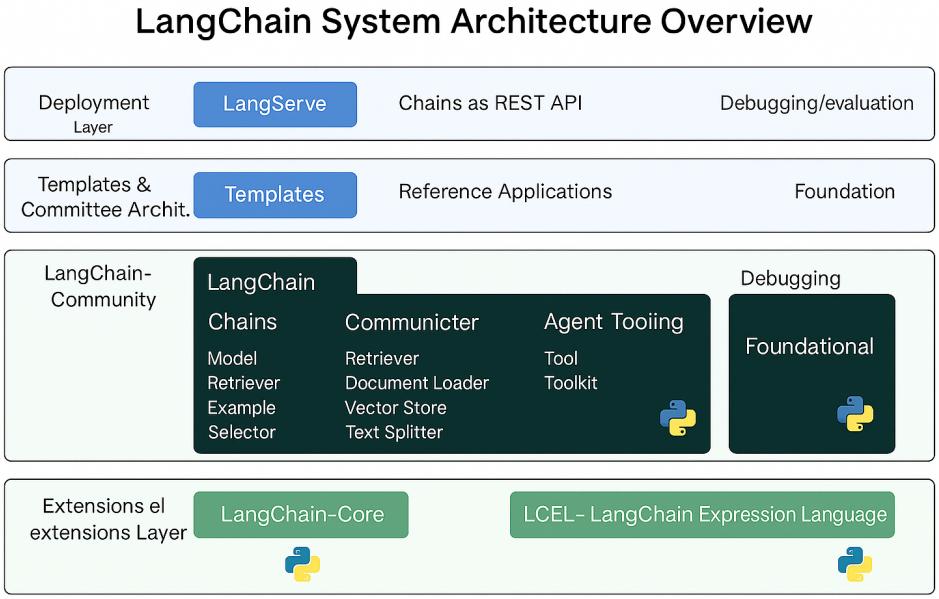


Figure 13: A comprehensive overview of the system architecture of LangChain [111].

Moreover, LangGraph [228] is a key extension within the LangChain ecosystem, specifically designed for building stateful, multi-agent dynamic workflows [140, 243, 111]. It addresses the gaps in LangChain for complex cyclic processes and real-time interaction scenarios, offering developers a more flexible and efficient solution. Core features of LangGraph include cyclic graph support (loops and conditional branches between nodes), fine-grained state control (e.g., dynamic graph modification), and built-in persistence capabilities (e.g., task interruption recovery and human intervention). These features of LangGraph empower developers to effortlessly implement AI applications requiring iterative decision-making (such as adaptive RAG pipelines and multi-agent automation systems), while seamlessly integrating with the LangChain ecosystem.

3.2.5 LlamaIndex (From Jerry Liu @LlamaIndex)

LlamaIndex[12] [88, 244] is a specialized framework designed to optimize the development of Retrieval-Augmented Generation (RAG) applications [46, 139, 208, 229], excelling in multiple technical dimensions. Compared to well-known projects like Langchain [111, 152, 111], LlamaIndex distinguishes itself through its focused domain optimization and innovative design philosophy, offering users a more efficient and specialized development experience for RAG applications. LlamaIndex demonstrates exceptional versatility in handling diverse data formats.

[11]JavaScript: https://web.developers.google.cn/javascript.
[12]LlamaIndex: https://docs.llamaindex.ai/en/stable/.

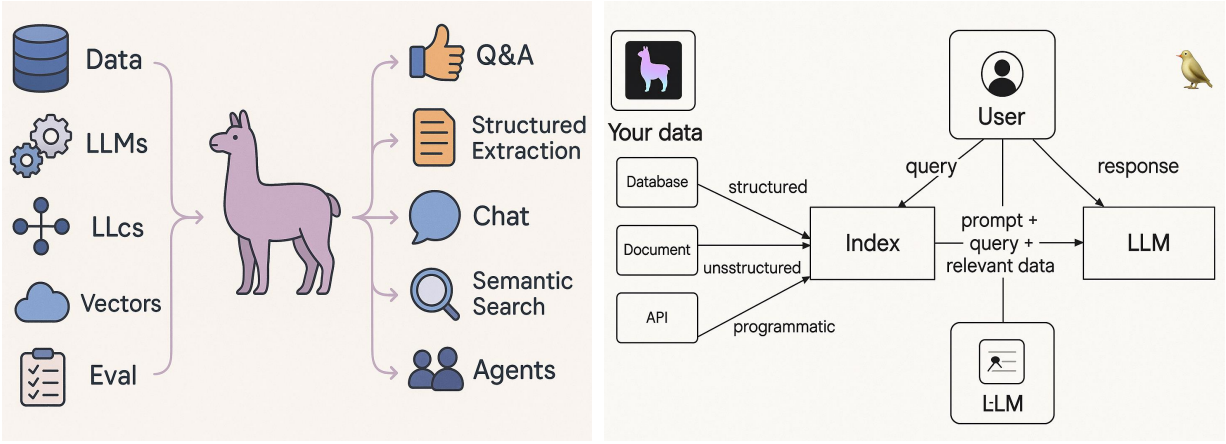


Figure 14: LlamaIndex architecture overview: (1) Left: Supported application scenarios of LlamaIndex; (2) Right: Data processing workflow of LlamaIndex.

As seen in the left side of Figure 14, LlamaIndex's data processing pipeline demonstrates its capability to ingest diverse data sources including databases, structured documents, and unstructured APIs [144, 150, 229]. This framework transforms raw data through LLM processing and vectorization, creating optimized representations for retrieval tasks. This visualization highlights the system's robust data ingestion and preprocessing capabilities that form the foundation for effective RAG applications. The right side of Figure 14 showcases LlamaIndex's supported use cases, ranging from Q&A systems and structured data extraction to semantic search and agent-based applications. This demonstrates the framework's versatility in addressing various NLP tasks while maintaining its specialized focus on retrieval-augmented generation scenarios. The separation of these components reflects LlamaIndex's modular architecture, allowing developers to implement specific functionalities while benefiting from the framework's optimized RAG core [138, 229, 244].

And its key features and advantages-including Robust Data Ingestion & Preprocessing [244, 144, 150], Optimized Indexing & Query Architecture [46, 138, 244], Native Multimodal Support [79, 162], Production-Grade Scalability [8, 56], Modular & Extensible Design [94, 111], Ecosystem Integration [157, 158, 123] can be concluded as follows:

First, LlamaIndex [244] excels in data ingestion and preprocessing. It not only supports a wide range of structured and unstructured data formats but, more importantly, ensures high-quality data encoding into LLM memory through flexible text splitting and vectorization mechanisms [144, 150, 229]. This lays a solid foundation for contextual understanding during the generation phase.

At the same time, LlamaIndex provides a rich selection of indexing data structures and query strategies, enabling developers to fully leverage scenario-specific efficiency advantages and achieve high-performance semantic retrieval. Such targeted optimizations are undoubtedly critical for RAG applications [46, 138].

Another notable highlight is LlamaIndex's native support for multimodal data (such as images and videos) [79, 162]. By integrating with leading vision-language models, it can introduce rich cross-modal context into the RAG generation process, adding new dimensions to the output. Undoubtedly, this will pave the way for numerous innovative applications.

Beyond core data management capabilities [245], LlamaIndex also emphasizes engineering best practices for RAG application development. It offers advanced features such as parallelized queries and Dask-based distributed computing support, significantly improving data processing efficiency and laying the groundwork for large-scale production deployment [8, 56].

From an architectural perspective, LlamaIndex adheres to modular and extensible design principles. Its

flexible plugin system allows developers to easily incorporate custom data loaders, text splitters, and vector indexing modules, catering to diverse and personalized requirements across different scenarios [94, 111].

Additionally, seamless integration with the open-source ecosystem is an inherent strength of LlamaIndex. It provides out-of-the-box compatibility with popular tools and frameworks like Hugging Face[13] and FAISS[14] [158, 159], enabling users to effortlessly leverage cutting-edge AI/ML capabilities and accelerate the development of innovative solutions.

As a professional-grade tool dedicated to RAG applications [46, 139, 208], LlamaIndex has become an ideal complement to general-purpose frameworks like Langchain. Developers can now freely choose between LlamaIndex's optimized, high-efficiency approach and Langchain's versatile, flexible paradigm based on their specific needs, maximizing both development efficiency and product quality.

As an evolving framework, LlamaIndex [244] continues to enhance its modeling capabilities for complex scenarios while integrating emerging advancements in LLM architectures and RAG methodologies. Future development focuses on intelligent auto-optimization features and expanded reference implementations to accelerate enterprise adoption.

3.2.6 AutoChain (From Forethought-AI)

AutoChain[15] [246] is a lightweight yet extensible framework proposed by Forethought-AI[16]. It builds upon the foundations of LangChain [111, 140] and AutoGPT [113], offering developers a more efficient and flexible approach to building conversational AI agents. AutoChain's core philosophy revolves around three key principles: simplicity, customizability, and automation.

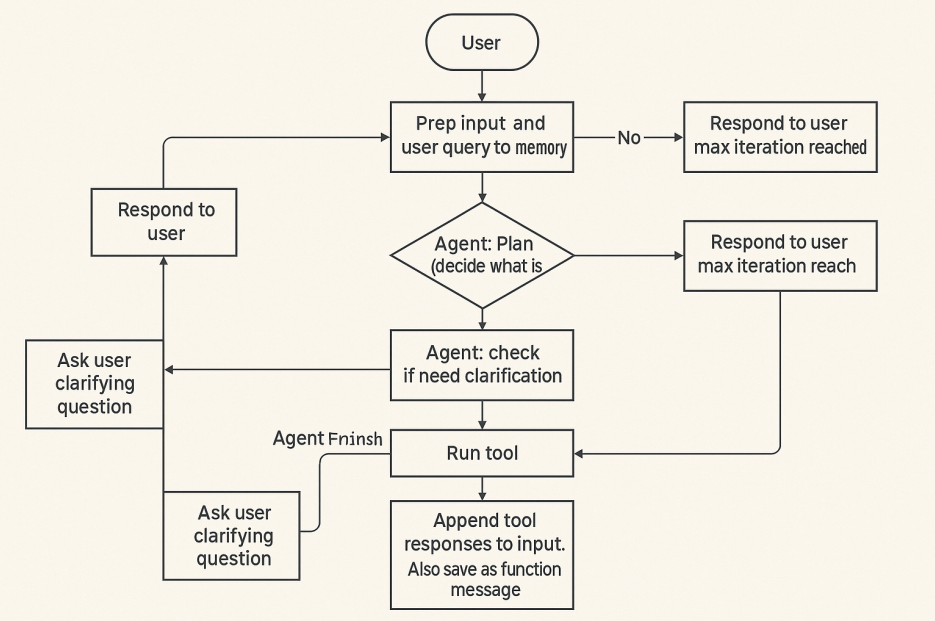


Figure 15: The Architecture Framework of AutoChain [246]: Agent Workflow with Conditional Branching, External Tool Calls, and Termination Control.

As seen in Figure 15, the framework deliberately maintains a minimalist architecture to reduce cognitive overhead, abstracting fundamental LLM application workflows into intuitive building blocks. The workflow

[13]Hugging Face: https://huggingface.co/.
[14]FAISS: https://faiss.ai/.
[15]AutoChain Github: https://github.com/Forethought-Technologies/AutoChain.
[16]AutoChain-AI: https://autochain.forethought.ai/.

of AutoChain [246], that shown in Figure 15, facilitates automated interactions between an intelligent agent and a user. The process begins by receiving and storing the user's query. The agent then evaluates whether to continue the interaction based on predefined conditions. If the maximum iteration limit is reached, it terminates with a final response; otherwise, the agent proceeds to the *Plan* phase to determine the next action (e.g., generating a response or requesting clarification). When external tools are required, the system checks if additional information is needed. If not, it executes the tool and appends the results to the input stream, storing them as a *Function Message*. The loop is controlled through binary decisions between *AgentAction* and *AgentFinish*, ultimately producing the agent's output [92, 247, 248].

Unlike more complex alternatives, AutoChain [246] provides unparalleled customization through pluggable tools, data sources, and decision modules, enabling developers to create tailored solutions for unique use cases. This "embrace differentiation" approach is complemented by built-in conversation simulation capabilities that automate agent evaluation across diverse interaction scenarios, significantly accelerating development cycles.

AutoChain [246]'s balanced design caters to multiple user groups: beginners benefit from its gentle learning curve when creating basic dialogue agents [51, 52, 105], experienced LangChain users appreciate its familiar-but-simpler concepts for rapid prototyping, while AI researchers value its clean-slate extensibility for developing novel paradigms [31, 113].

3.2.7 Flowise AI (From FlowiseAI Inc.)

Flowise AI[17] [249] emerges as an innovative open-source platform that significantly lowers the barriers to building LLM-based [37, 250] applications through its no-code, drag-and-drop visual interface[18]. Unlike traditional coding-intensive frameworks, Flowise enables developers to construct sophisticated LLM workflows by simply connecting pre-built components, making AI application development accessible to non-programmers while maintaining professional-grade capabilities.

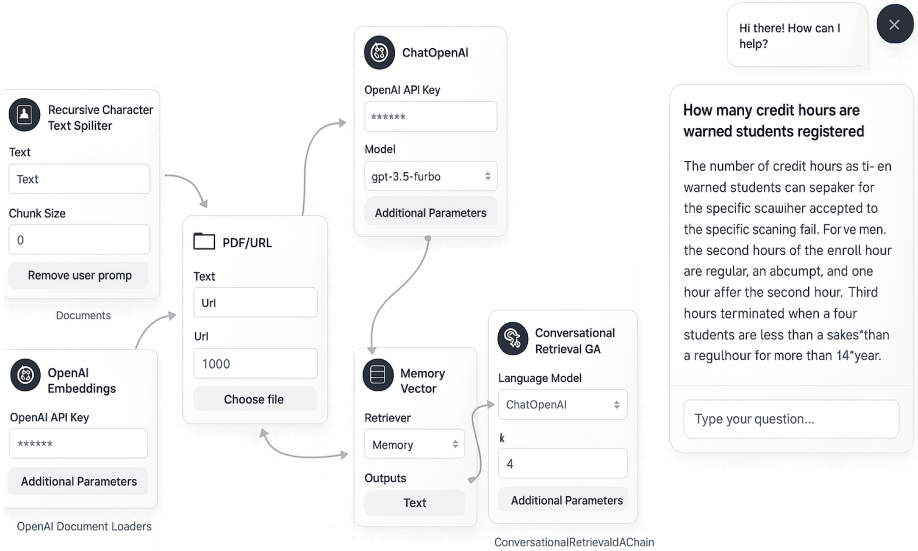


Figure 16: Automatic conversational retrieval architecture of Flowise AI.

As shown in Figure 16, this workflow demonstrates a typical conversational retrieval QA pipeline of

[17]Flowise AI Github: https://github.com/FlowiseAI/Flowise.
[18]Flowise AI: https://flowiseai.com/.

Flowise-AI. The process begins with document ingestion using a PDF loader and text splitter, followed by generating vector embeddings through OpenAI's embedding API. These embeddings are stored in an in-memory vector store, enabling efficient semantic search. A user query is then processed by a language model (ChatOpenAI) [156, 43], which interacts with the memory retriever to fetch relevant context and return accurate answers. Flowise-AI allows researchers and developers to build customized LLM applications without extensive programming knowledge, making it ideal for rapid prototyping and production-ready deployments [29, 37, 250].

The most innovative aspect of Flowise lies in its deep integration with LangChain's powerful framework, exposing its full functionality - including LLM orchestration, chained operations, and data augmentation - through intuitive visual modules. The platform offers native compatibility with leading LLM providers (Anthropic, OpenAI, Cohere) [45, 199, 201] and data ecosystems (Pandas, SQL, Web APIs) [59, 157, 158], ensuring users can leverage cutting-edge AI capabilities without infrastructure complexities.

Notably, Flowise combines this accessibility with enterprise-ready extensibility, featuring API support and embeddable deployment options for integration into diverse software environments. This unique combination of no-code simplicity, LangChain-powered sophistication, and ecosystem interoperability positions Flowise as a pivotal tool in the democratization of LLM technologies, enabling both individuals and organizations to rapidly develop and deploy AI solutions [37, 250].

3.3 Comparisons of AI-Driven Automation Tools

The systematic comparison presented in Table 9 reveals fundamental architectural trade-offs among contemporary AI-driven automation tools, highlighting three critical dimensions of differentiation in their design philosophies.

First, the memory management spectrum demonstrates a clear progression from transient task-specific implementations (OpenManus) [114] to sophisticated hybrid approaches combining long-term user context (Manus) [117] with external knowledge integration (LangChain and LangGraph) [111, 228]. This evolution reflects the growing industry demand for systems that balance operational efficiency with contextual awareness, particularly evident in OWL's adaptive incremental learning mechanism that bridges internal and external data sources [120, 167].

Second, execution flow patterns diverge significantly between sequential-linear models (like UiPath AI Fabric [106, 107]) and parallel-coordinated approaches (like OWL [120]), with Manus occupying a unique middle ground through its hybrid sequential-parallel execution with feedback integration. This architectural choice directly impacts each tool's suitability for different workflow complexities, where LangChain's sequential NLP-focused processing contrasts sharply with OWL's multi-agent parallel coordination for industrial-scale automation.

Third, the integration capabilities expose a fundamental tension between specialization and generalization. While LangChain [111] maintains tight focus on NLP and AI workflow integration, UiPath AI Fabric [106] prioritizes depth in enterprise tool connectivity, and OWL [120] emphasizes cross-platform interoperability. Manus and OpenManus [114, 117] demonstrate opposing approaches to customization - the former through extensive custom tool integration and the latter via modular plug-and-play components - representing distinct solutions to the flexibility-scalability trade-off.

These comparative insights suggest that modern automation platforms are evolving along two orthogonal axes: (1) from single-agent to collaborative multi-agent automation architectures [91, 129, 136, 251], and (2) from rigid workflow enforcement to adaptive process orchestration [102, 122, 217]. The emerging generation of tools (exemplified by Manus and OWL [120]) increasingly combines cognitive flexibility with industrial-grade reliability, though significant challenges remain in achieving seamless interoperability across these divergent architectural paradigms.

Table 9: Comparative Analysis of AI-Driven Automation Tools

Aspects \ Tools	**Manus** [117, 119]	**OpenManus** [114]	**OWL** [120]	**LangChain** [111]	**AI-Driven UiPath** [106]
Task Complexity	High-Multi-agent, planning and execution	Moderate-Simplified modular design	High-Multi-agent collaboration	Moderate-Focus on NLP and workflows	Moderate-Focused on RPA with AI
Memory	Long-term with user history	Short-term task-specific memory	Incremental, integrates external data	External knowledge integration	Task-specific memory with reusable workflows
Architecture	Multi-agent automation system, task decomposition	Modular plug-and-play framework	Multi-agent with cross-platform support	Rule-based with AI-driven workflows	Hybrid-RPA with AI-powered tools
Execution Flow	Sequential and parallel with feedback loops	Sequential, task division, execution	Parallel execution, multi-agent coordination	Sequential task execution	Sequential, linear task execution
Tool Integration	Extensive custom tool integrations	High flexibility in tool choice	Cross-platform tool integration	Focused on NLP and AI workflows	Deep integration with enterprise tools
Memory Management	Task-dependent and user-specific memory	Task-dependent, short-term memory	Adaptive, incremental learning	External, dynamic memory	Task-based, context-driven memory

3.4 Scenario-Based Comparative Analysis of AI-Driven Automation Tools

To provide a more concrete understanding of the strengths and limitations of different AI-driven automation tools, we further conduct a scenario-based comparative analysis, compared with Section 3.3. This evaluation examines how each tool performs under three representative use cases: complex workflow automation, and enterprise-scale RPA integration.

The scenario-based comparison in Table 10 reveals critical trade-offs in tool selection across different automation contexts. For complex workflows requiring dynamic adaptation (Scenario Z1), systems like Manus [117] and OWL [120] demonstrate superior capabilities in handling iterative refinement and parallel execution, though they demand greater technical expertise for customization. In contrast, tools like UiPath AI [106] offer immediate productivity gains for standardized enterprise processes but sacrifice flexibility in dealing with unstructured exceptions.

Enterprise integration scenarios (Scenario Z2) expose divergent approaches to bridging traditional RPA with modern AI capabilities. While specialized tools like LlamaIndex deliver exceptional performance for document-centric workflows, their narrow optimization comes at the expense of broader automation applicability. The comparison underscores an emerging industry need for platforms that can simultaneously deliver UiPath's [106] enterprise readiness, Flowise's [249] accessibility, and LangChain's [111] technical extensibility without compromising performance in production environments. These insights provide practitioners with a structured framework for evaluating tools against their specific operational requirements and technical constraints.

Table 10: Scenario-Based Comparative Analysis of AI-Driven Automation Tools

Scenario	Tool	Strengths	Limitations
Z1: Complex Workflow Automation	Manus [117]	Advanced task decomposition; supports iterative refinement via feedback loops	High setup complexity; requires custom agent tuning
	OWL [120]	Multi-agent coordination excels at parallel subtask execution	Limited native document processing tools
	LangChain [111]	Strong NLP integration for text-heavy workflows	Weak handling of non-textual data (e.g., forms, tables)
Z2: Enterprise -Scale RPA Integration	UiPath AI [106]	Native integration with legacy RPA systems and ERP platforms	Limited AI agent customization capabilities
	Flowise AI [249]	Visual no-code interface accelerates workflow design	Scalability challenges in high-volume production environments
	LlamaIndex [244]	Optimized for document-intensive RPA with RAG capabilities	Narrow focus on retrieval-augmented tasks limits generalizability
	LangChain [111]	Modular architecture supports hybrid RPA-AI pipelines	Requires significant development effort for enterprise integration

3.5 Summary

In this section, we present a systematic exploration of cutting-edge AI-driven automation technologies and their real-world applications. Our analysis begins by mapping the field's progression from basic rule-based systems to advanced cognitive architectures capable of dynamic reasoning and multi-agent collaboration. We examine how modern automation frameworks achieve unprecedented flexibility through innovative approaches to task planning, memory handling, and tool integration. The discussion covers various system implementations that demonstrate different strategies for workflow execution, from linear processing to sophisticated agent coordination models.

We also provide detailed comparisons of leading automation tools, revealing fundamental design choices and their practical implications. Our architectural analysis uncovers how contemporary systems balance performance with adaptability, while maintaining robust operation across diverse application scenarios. Through this comprehensive review, we illuminate the remarkable progress in AI-driven automation while establishing a clear framework for understanding current technological capabilities. The findings demonstrate how these systems are evolving from simple task automation to comprehensive problem-solving platforms.

4 Challenges in AI-Driven Automation

4.1 Existing Limitations

However, despite the remarkable progress in AI-driven automation, several challenges continue to hinder the widespread deployment of these technologies [24, 64, 252]. From data quality and governance to model interpretability [64, 67], and from security [73, 183] to robustness [84, 136], the technical barriers of AI-driven automation significantly impact its progress. As shown in Figure 17, we systematically categorize the key technical barriers to AI-driven automation into four interdependent dimensions: (1) *data-related challenges*, (2) *model limitations*, (3) *implementation barriers*, and (4) *security concerns*. This taxonomy reveals how fundamental issues in data quality and governance propagate through model development to system deployment [66, 70, 75]. The following discussion analyzes each category through both technical and operational lenses.

First and foremost, data quality and governance represent one of the primary challenges faced by AI-driven automation [33, 66, 70]. The performance of AI models is highly dependent on the quality and accuracy of input data. However, in the real world, data often suffers from noise, missing values, inconsistency, or

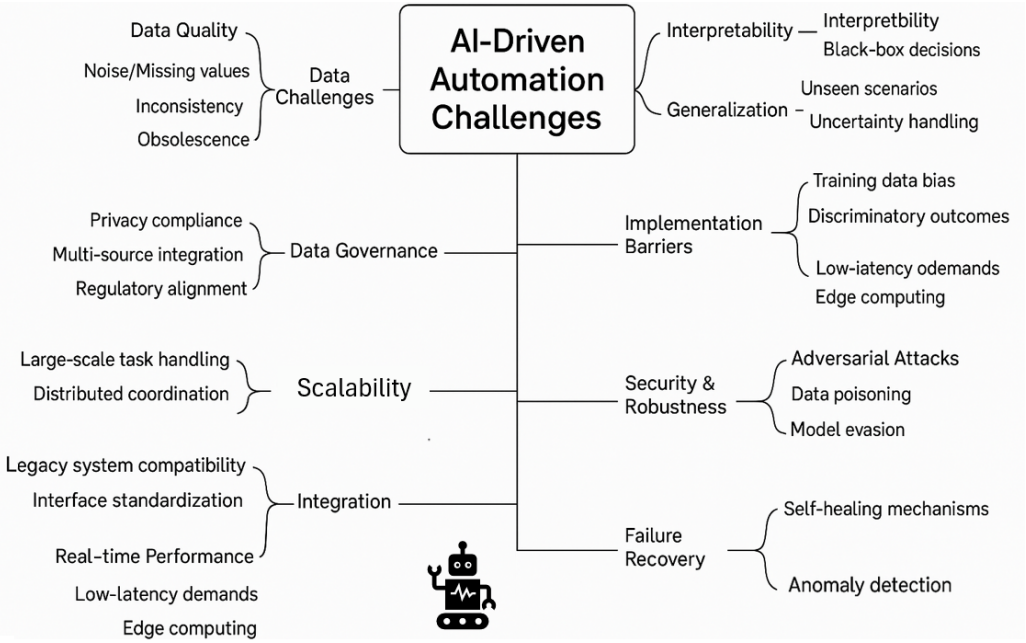


Figure 17: The primary categories of challenges in the development and deployment of AI-driven automation technologies.

obsolescence. This is particularly challenging in complex application scenarios, where ensuring the accuracy of data cleaning, integration, and processing becomes a critical issue. Additionally, in the realm of data governance [26, 182, 183], ensuring data privacy and security is a pressing concern. Automation systems need to process vast amounts of complex and diverse data sources while ensuring compliance with data privacy regulations, which places higher demands on technical infrastructure.

In addition to data governance, model interpretability and transparency are major challenges in current AI-driven automation technologies [7, 64, 67]. While deep learning and other complex algorithms have achieved significant success in specific tasks, their "black-box" nature makes it difficult to explain the task execution process. In high-risk fields, such as finance [3, 17, 24] and healthcare [13, 57, 14, 219], the lack of transparency in AI systems' decisions can lead to traceability issues, thereby reducing trust in these systems. Therefore, improving the interpretability of AI systems, making their automatic production processes understandable to human users and developers, becomes crucial to enhancing transparency and fairness in automation systems [64, 67].

Apart from interpretability, the generalization ability of models is also an important factor influencing AI-driven automation [39, 84, 136]. Many AI systems perform well in specific environments but may exhibit poor performance when faced with new, unseen situations. The actual applications of automation systems are often filled with uncertainties, and enhancing the adaptability of models to handle diverse human living needs is an urgent challenge that AI technology must address. To achieve this, AI models must be capable of processing various data patterns and reasoning effectively in complex and uncertain environments [39, 84].

Another technical difficulty in AI-driven automation is system scalability. As automation applications expand, ensuring that systems can handle larger-scale tasks while maintaining efficiency and low latency becomes a significant challenge [8, 56, 65]. Especially in cloud computing and distributed environments, coordinating the collaboration of multiple AI models and managing resources effectively under dynamic loads remains a key issue in system design. This requires engineers to optimize resource management, load balancing, and fault tolerance within the system architecture [8, 56].

Moreover, issues related to algorithm fairness and bias are crucial in the development of AI technologies

[24, 64, 252]. AI systems' decisions heavily rely on training data, which may contain societal, cultural, and historical biases. These biases may lead to unfair or discriminatory decisions when automation is applied. In sensitive areas such as recruitment, credit evaluation, and judicial decisions, algorithmic bias can have profound negative consequences for individuals and society. Therefore, eliminating algorithmic bias and ensuring fairness in AI decisions remain key issues that must be prioritized in current AI research [64, 252].

In terms of technical implementation, integration of AI-driven automation systems is another critical challenge [55, 80, 106]. AI-driven automation systems often need to collaborate with existing enterprise systems, software tools, hardware devices, and other automation technologies. Ensuring seamless integration of these different technological stacks and smooth data flow between them is a major issue in system design. Challenges in this area include compatibility between different platforms, standardization of data interfaces, and other related issues, which directly affect the deployment and maintenance of AI-driven automation systems [55, 80].

At the same time, the real-time performance and processing capabilities of AI systems [19, 184, 186] must be considered in automation applications. In many practical scenarios, automation decisions need to be made within milliseconds or seconds, placing high demands on the system's computational power. Optimizing the efficiency of algorithms, reducing latency, and balancing computational resources and response speed, especially in big data and edge computing environments, have become critical concerns.

Finally, security and robustness [26, 73, 183] are still key obstacles in the application of AI-driven automation technologies. Due to the complexity of AI systems, they often become targets for malicious attacks. For example, hackers might manipulate data or perform model attacks to disrupt the human producing process of automation systems. Therefore, enhancing the security and robustness of AI systems to ensure stable operation in the face of external attacks and internal anomalies is a prerequisite for the success of AI automation applications.

In conclusion, while AI-driven automation technologies hold immense potential to drive intelligent transformation across industries, they still face a series of challenges in technical implementation [66, 70, 75]. Issues such as data quality and governance, model interpretability, system scalability, and algorithm fairness require continuous technological innovation and interdisciplinary collaboration to overcome [64, 252]. As these technical difficulties are gradually addressed, AI-driven automation is expected to further enhance productivity and efficiency across various sectors, bringing profound changes to society.

4.2 Privacy and Security Compliance in AI-Driven Automation Systems

As AI-driven automation systems are increasingly deployed across sensitive domains such as healthcare, finance, and energy, ensuring compliance with privacy and security regulations like GDPR [19] becomes essential. This section outlines key mechanisms adopted by representative tools to address data protection, encryption, and authentication challenges.

4.2.1 Compliance Standards and Data Protection Mechanisms

GDPR Compliance. Tools such as *Med-Bot* [57] and *TradingGPT* [16] incorporate GDPR-aligned features including dynamic consent tracking, and data anonymization. These systems enforce data minimization and transparency through anonymous processing and user-controlled data retention policies.

HIPAA-Compliant Architectures. In healthcare, platforms like *MedAgents* [12] and GPT-based medical chatbots [57] utilize HIPAA-compliant data pipelines. These systems support encrypted data storage and transmission (e.g., AES-256[20])

[19]GDPR: https://gdpr-info.eu/art-32-gdpr/.
[20]AES-Standard: https://www.nist.gov/publications/advanced-encryption-standard-aes.

4.2.2 Data Isolation, Encryption, and Authentication

To address potential risks to data integrity and confidentiality, the tools discussed employ robust data isolation techniques. For instance, in multi-agent cooperative models like OWL's energy management system [120], distributed edge AI environments utilize hardware-based isolation to protect data at each edge node. This prevents unauthorized access to data generated by the agents, ensuring that only trusted systems have access to sensitive operational data. Meanwhile, the use of end-to-end encryption is standard across many AI-driven automation platforms. For example, systems like SuperAGI [116] and OWL [120] leverage advanced encryption protocols such as AES-256 to protect data during storage and transmission. This ensures that even if the data is intercepted during communication, it remains unreadable to unauthorized entities. Despite these robust compliance and security mechanisms, challenges remain in ensuring full interoperability and scalability across systems that span multiple industries. Tools like AutoGPT [113] and WorkGPT [217], which operate across different domains, need to account for varying compliance standards in each sector.

In summary, contemporary AI-driven automation systems increasingly embed regulatory compliance as a core architectural principle, integrating mechanisms such as auditability, encryption, access control, and federated data isolation. Addressing these evolving requirements will be pivotal for the sustained deployment of AI solutions across regulated environments, particularly as automation extends into multi-agent, edge-centric, and globalized operational contexts.

4.3 Summary

In this section, we systematically examine the critical challenges currently facing AI-driven automation technologies through a structured analysis framework. Our investigation reveals how fundamental issues in data quality and governance create cascading effects throughout the entire automation pipeline, from model development to system deployment. The discussion highlights the complex interplay between technical limitations and operational constraints that collectively impact real-world implementation.

We also demonstrated that the path toward reliable and trustworthy automation systems requires addressing multiple interdependent factors. The challenges span across data management, model interpretability, system robustness, and security considerations. Each presenting unique technical hurdles while simultaneously influencing other aspects of system performance. The examination provides valuable insights into the current technological barriers while establishing clear directions for future research and development.

5 Future Directions and Opportunities

The rapid evolution of AI-driven automation technologies has unlocked unprecedented opportunities for transforming industries, yet significant gaps remain in achieving robust, ethical, and scalable implementations [1, 3, 75, 252]. Building on the challenges outlined above, here we explore promising research directions and emerging opportunities that could redefine the trajectory of AI-driven automation.

Specifically, these directions span technical innovations [72, 78, 175], interdisciplinary synergies [16, 32, 63], regulatory frameworks [17, 64, 180], and novel application paradigms [74, 235, 251], aiming to address current limitations while amplifying the societal and economic impact of automation technologies. As shown in Figure 18, we present a comprehensive taxonomy of future research directions in AI-driven automation, systematically organized across ten key dimensions ranging from foundational cognitive architectures to industry-specific implementations. In the following subsections, we provide detailed analysis of each direction, beginning with core advances in cognitive systems.

5.1 Enhancing Cognitive Architectures for Dynamic Adaptation

A critical frontier lies in advancing AI-driven automation systems to dynamically adapt to evolving environments [97, 142, 253]. Current architectures, while increasingly sophisticated, remain constrained by



Figure 18: An Overview of Future Trends and Opportunities in AI-Driven Automation Tools and Systems.

rigid workflows or limited contextual awareness. Future research should prioritize self-architecting systems capable of autonomously selecting optimal execution paradigms (sequential, parallel, or hybrid) based on real-time workflow analysis. For instance, integrating reinforcement learning [85, 168] with meta-learning [60, 230] frameworks could enable systems to dynamically reconfigure task decomposition strategies or switch between rule-based and LLM-driven reasoning. Emerging tools like AutoGPT [113] hint at this potential but lack formal mechanisms for evaluating architectural trade-offs in real time.

Another promising direction involves multi-modal cognitive integration [79, 174], where these systems seamlessly process and correlate diverse data types-text, speech, images, and sensor inputs. This demands advancements in cross-modal attention mechanisms and unified knowledge representation frameworks. For example, combining transformer-based language models [156, 43] with graph neural networks [164, 165, 166] could enable systems to parse technical manuals, interpret equipment sensor data, and generate maintenance protocols within a single workflow. Such architectures would be transformative in manufacturing and healthcare, where decisions often hinge on heterogeneous data streams.

5.2 Advancing Human-AI Collaboration Frameworks

The next generation of AI-driven automation tools will transcend conventional "human-in-the-loop" models [254] to achieve context-aware symbiosis. Current AI-driven automation systems like MetaGPT [60] and OWL [120] demonstrate early-stage collaborative capabilities but lack nuanced understanding of human expertise levels or situational constraints. Future systems could employ adaptive autonomy controllers that modulate AI involvement based on:

- Operator skill profiling [146, 175]: Leveraging user interaction histories to personalize assistance levels.
- Task criticality assessment [12, 57]: Automatically escalating decisions to humans in high-risk scenarios (e.g., medical diagnostics).

- Cognitive load optimization [213, 255]: Balancing automation depth with human oversight requirements to prevent decision fatigue.

Innovative interfaces will play a pivotal role here. Augmented reality (AR) [49, 50] overlays, as seen in experimental platforms like DeChen's HMI solutions [220], could evolve into AI-guided procedural assistants. Imagine maintenance technicians receiving real-time, context-sensitive AR instructions generated by LLMs analyzing equipment manuals and historical repair data [128, 196]. Such systems would require ultra-low-latency architectures (less than 10ms response times) and robust anomaly detection to prevent hazardous misinterpretations.

5.3 Bridging the Simulation-to-Reality Gap

A persistent challenge in industrial automation is the discrepancy between simulated environments and real-world deployment [22, 176, 256]. Emerging digital twin technologies offer a pathway to address this through:

- Physics-informed neural networks [100, 230]: Embedding domain-specific constraints (e.g., mechanical stress limits) into AI models to improve simulation fidelity.
- Continuous twin updating [16, 32]: Implementing bidirectional data flows where real-world sensor feedback refines digital models in real time.
- Federated learning architectures [65, 130]: Enabling collaborative model training across distributed digital twins while preserving data privacy.

For example, in energy grid management, a federation of digital twins could simulate regional demand patterns while OWL-like multi-agent automation systems optimize power distribution [120]. This approach would demand breakthroughs in distributed optimization algorithms and secure data-sharing protocols to prevent adversarial attacks on interconnected twins.

5.4 Ethical AI and Regulatory Compliance by Design

As automation permeates high-stakes domains, embedding ethical guardrails into system architectures becomes imperative [64, 67, 17]. Current efforts, such as differential privacy in BD HemoSphere Alta [218, 220], represent isolated solutions rather than systemic approaches. Future frameworks should integrate:

- Dynamic compliance engines [14, 34]: Neural-symbolic systems that automatically map regulatory requirements (e.g., GDPR[21], HIPAA[22]) to workflow constraints.
- Bias mitigation pipelines [71, 73]: Multi-stage architectures where data preprocessing, model training, and output validation layers each incorporate fairness checks.
- Explainability-as-a-service [64, 67]: Modular interfaces that generate audit trails and rationale explanations tailored to different stakeholders (technicians, regulators, end-users).

A groundbreaking opportunity lies in developing global regulatory sandboxes-shared testing environments where governments and corporations collaboratively validate compliance mechanisms [35]. These sandboxes could utilize blockchain-based smart contracts to enforce transparency in automated producing processes, particularly in finance and healthcare.

[21] General Data Protection Regulation (GDPR): https://gdpr-info.eu/.
[22] Health Insurance Portability and Accountability Act (HIPAA): https://www.cloudflare-cn.com/learning/privacy/what-is-hipaa-compliance/.

5.5 Sustainable Automation through Energy-Efficient AI

The computational demands of modern AI systems, particularly LLMs, pose significant environmental challenges [73, 198]. Future research must prioritize green automation strategies:

- Sparse model architectures [131, 230]: Leveraging techniques like mixture-of-experts (MoE) [94, 131] to reduce inference costs while maintaining performance.
- Hardware-algorithm co-design [11, 216]: Developing specialized chips (e.g., neuromorphic processors [190, 257]) optimized for common automation tasks like sensor fusion or NLP.
- Carbon-aware scheduling [73, 198]: Dynamically shifting computational loads to regions/times with renewable energy surplus.

Experimental platforms like OWL [120]'s edge-computing modules demonstrate early progress, but broader adoption requires standardized metrics for measuring automation's carbon footprint [35]. Integrating lifecycle assessment tools into AI development pipelines could enable "sustainability-by-design" in next-generation systems [6, 75].

5.6 Cross-Domain Generalization and Open-World Learning

Most current AI-driven automation tools excel in narrow domains but struggle with unseen scenarios-a critical limitation given the unpredictable nature of real-world environments [175, 97]. Addressing these demands:

- Meta-learning frameworks [142, 230]: Systems that rapidly adapt to new tasks by leveraging knowledge from related domains. For instance, a document processing pipeline trained on legal contracts could generalize to medical records through few-shot learning.
- Causal reasoning integration [175, 97]: Moving beyond correlation-based models to systems that infer underlying causal relationships, enhancing robustness in dynamic settings.
- Open-world validation benchmarks [32, 32, 74]: Developing testing environments that simulate novel edge cases, such as equipment failures or unprecedented market shifts.

The fusion of LLMs with simulation engines presents a unique opportunity here. Imagine training AI agents in synthetic "automation gyms" that procedurally generate challenges ranging from supply chain disruptions to cyberattacks, fostering generalized problem-solving capabilities.

5.7 Democratization through Low-Code/No-Code Evolution

While platforms like Zapier have democratized automation development, significant barriers remain for non-technical users [30, 29, 38]. Future no-code systems could incorporate:

- Intent-based programming [37, 30, 250]: Allowing users to describe goals in natural language while AI handles implementation details.
- Self-debugging workflows [28, 41]: Automated root-cause analysis and correction of faulty automation scripts.
- Collaborative AI tutors [36, 42]: Context-aware assistants that guide users through complex workflow design via interactive dialogues.

A radical innovation would be self-assembling automation ecosystems, where AI agents negotiate API integrations and resolve compatibility issues autonomously [111, 111]. This would require advances in semantic protocol translation and distributed consensus mechanisms, building upon current efforts in tools like LangChain [111, 112].

5.8 Security and Resilience in Hyperconnected Systems

As AI-driven automation systems grow increasingly interconnected, novel attack vectors emerge. Future safeguards must adopt a multi-layered defense paradigm:

Homomorphic encryption [73, 71]: Enabling secure data processing in untrusted environments (e.g., cloud-based RPA).

Adversarial robustness certification [136, 258]: Formal methods to guarantee system behavior under worst-case scenarios.

Self-healing architectures [39]: Systems that automatically isolate compromised components and regenerate clean states using immutable ledger technologies.

The integration of quantum-resistant cryptography and AI-driven threat detection could create next-generation shields for critical infrastructure automation. For instance, power grid controllers might employ neural networks trained on synthetic attack data to preemptively neutralize threats.

5.9 Toward Artificial General Intelligence (AGI) in Automation

While current systems excel at specific tasks, the holy grail remains context-aware AGI capable of open-ended reasoning. Incremental steps toward this vision include:

- Recursive self-improvement [39]: Systems that autonomously refine their architectures through evolutionary algorithms.
- Embodied AI integration [155, 176, 235]: Combining software automation with robotic systems that learn through physical interaction.
- Cross-modal knowledge transfer [46, 133, 169]: Enabling insights gained in one domain (e.g., logistics optimization) to inform decisions in unrelated fields (e.g., patient scheduling).

Ethical considerations here are paramount. The development of AGI-powered automation demands international governance frameworks to prevent uncontrolled recursive self-enhancement scenarios.

5.10 Industry-Specific Revolution

Finally, domain-specific innovations will drive automation's next wave:

- Healthcare [13, 57, 14]: Federated learning systems that enable collaborative diagnosis across hospitals without sharing sensitive patient data.
- Manufacturing [25, 27, 68, 211]: Swarm robotics coordinated through physics-aware AI models for adaptive production lines.
- Finance [3, 103, 17, 32]: Real-time regulatory compliance engines using quantum machine learning to detect emerging market risks.

Each requires tailored solutions that balance technical capabilities with domain constraints-a challenge demanding close academia-industry collaboration.

Overall, the future of AI-driven automation lies not in isolated technological breakthroughs, but in the synergistic integration of cognitive architectures, ethical frameworks, and human-centric design. By addressing current limitations in adaptability, transparency, and sustainability, researchers can unlock the full potential of AI-driven automation as a force for equitable progress. Emerging paradigms-from self-built systems to AGI-assisted workflows-promise to redefine how organizations operate, but their success hinges on our ability to navigate technical complexities while upholding societal values. As the field advances, interdisciplinary collaboration will be crucial in transforming these visionary directions into tangible solutions that benefit industries and communities worldwide.

5.11 Summary

In this section, we explore the transformative potential and future directions of AI-driven automation across multiple dimensions. Our analysis reveals that the field is moving toward more adaptive, ethical, and human-centric systems, with key opportunities in cognitive architectures, human-AI collaboration, and industry-specific applications. We highlight how emerging technologies like digital twins, augmented reality interfaces, and quantum-resistant security protocols are creating new paradigms for intelligent automation. The discussion underscores the need for balanced progress that combines technical innovation with robust governance frameworks. As the field evolves, the most impactful advancements will likely emerge at the intersection of multiple disciplines, requiring close collaboration between researchers, practitioners, and policymakers to realize automation's full potential while addressing societal concerns.

6 Conclusion

The rapid evolution of AI-driven automation has ushered in a transformative era across industries, marked by unprecedented efficiency gains, enhanced task handling capabilities, and novel operational paradigms. Over the past decade, AI-driven automation has transitioned from rule-based systems to sophisticated cognitive architectures, yet the absence of a unified framework for categorizing methods, benchmarking performance, and addressing ethical risks has hindered both academic progress and industrial adoption. To bridge this gap, our work systematically reviews architectural innovations, functional paradigms, and real-world applications of AI-driven automation techniques across research areas and industries. Specifically, this survey has systematically examined the architectural, functional, and application-specific advancements in AI-driven automation, while systematically analyzing persistent challenges and emerging opportunities. Firstly, this survey begins by establishing several novel taxonomies to classify AI-driven automation tools into four architectural paradigms-rule-based, LLM-driven, multi-agent cooperative, and hybrid models-and further categorizes them by functionality, task execution mechanisms, and human-AI collaboration modes. Through comparative analysis of various AI-driven automation tools , emphasizing their applicability in domains such as healthcare, finance, and manufacturing. Finally, we provide a detailed summarization of challenges, including data governance gaps, model interpretability deficits, and scalability bottlenecks, provides a roadmap for addressing critical barriers to deployment. Additionally, we also discuss emerging trends, such as the integration of digital twins and neuro-symbolic reasoning, to forecast future research trajectories. By consolidating disparate advancements of AI-driven automation technique into a cohesive narrative, this survey not only clarifies the current advancements of AI-driven automation but also establishes foundational knowledge to guide responsible innovation, ensuring these transformative technologies align with societal needs and sustainable development goals.

Acknowledgements

This work is conducted by the first author, Weiqiang Jin, during his research at Xi'an Jiaotong University and his visiting period at The University of Hong Kong. The first author would like to express his deepest appreciation to Professor Hongyang Du from The University of Hong Kong for his expert and valuable insights, and to Professor Tagore from Xi'an Jiaotong University for his constant encouragement. The authors are particularly grateful to the editors and anonymous reviewers for their thorough evaluations, and insightful comments, which have significantly improved this work. The corresponding author of this paper is Biao Zhao.

Funding Statement

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Author Contributions

The authors confirm contributions to this paper as follows: Weiqiang Jin: Conceptualization, Methodology, Formal analysis, Writing-original draft, Writing-review & editing, Project administration, Investigation, Validation; Ningwei Wang (Equal. Contribution.): Software, Data curation, Visualization, Writing-review & editing; Lei Zhang: Formal analysis, Writing-review & editing; Xingwu Tian: Investigation, Visualization, Conceptualization, Project administration; Bohang Shi: Resources; Biao Zhao: Writing-review & editing, Supervision. All authors reviewed the results and approved the final version of the manuscript. All authors have reviewed the results and approved the final version of the manuscript.

Data Availability Statement

This work did not involve any associated code or datasets as it is primarily theoretical in nature. No data were generated or analyzed during this study.

Ethics Approval

Not applicable.

Conflicts of Interest

The authors declare no conflicts of interest regarding the publication of this study.

References

1. Executive Office of the President, "Artificial intelligence, automation, and the economy," The White House, Washington, D.C., Policy Report, 12 2016, accessed: 2023-12-24. [Online]. Available: https://obamawhitehouse.archives.gov/blog/2016/12/20/artificial-intelligence-automation-and-economy
2. S. Bhattacharyya, J. S. Banerjee, and D. De, Eds., *Confluence of Artificial Intelligence and Robotic Process Automation*, 1st ed., ser. Smart Innovation, Systems and Technologies. Springer Singapore, 2023, 54 b/w illustrations, 63 illustrations in colour.
3. Y. Shen and X. Zhang, "The impact of artificial intelligence on employment: the role of virtual agglomeration," *Humanities and Social Sciences Communications*, vol. 11, no. 1, p. 122, 01 2024.
4. G. Liu, P. Zhao, L. Liu, Y. Guo, H. Xiao, W. Lin, Y. Chai, Y. Han, S. Ren, H. Wang, X. Liang, W. Wang, T. Wu, L. Li, H. Wang, G. Xiong, Y. Liu, and H. Li, "Llm-powered gui agents in phone automation: Surveying progress and prospects," 2025. [Online]. Available: https://arxiv.org/abs/2504.19838
5. Z. Ke, F. Jiao, Y. Ming, X.-P. Nguyen, A. Xu, D. X. Long, M. Li, C. Qin, P. Wang, S. Savarese, C. Xiong, and S. Joty, "A survey of frontiers in llm reasoning: Inference scaling, learning to reason, and agentic systems," 2025. [Online]. Available: https://arxiv.org/abs/2504.09037
6. L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, p. 186345, March 2024.
7. L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, "Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 484–506, 2024.
8. S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena: A realistic web environment for building autonomous agents," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=oKn9c6ytLx
9. G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. Karlsson, J. Fu, and Y. Shi, "Autoagents: A framework for automatic agent generation," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, K. Larson, Ed. International Joint Conferences on Artificial Intelligence Organization, 8 2024, pp. 22–30, main Track. [Online]. Available: https://doi.org/10.24963/ijcai.2024/3
10. K. Pandya and M. Holia, "Automating customer service using langchain: Building custom open-source gpt chatbot for organizations," 2023. [Online]. Available: https://arxiv.org/abs/2310.05421
11. X. Zhang, H. Chen, Y. He, W. Niu, and Q. Li, "Automatically generating rules of malicious software packages via large language model," 2025. [Online]. Available: https://arxiv.org/abs/2504.17198
12. X. Tang, A. Zou, Z. Zhang, Z. Li, Y. Zhao, X. Zhang, A. Cohan, and M. Gerstein, "MedAgents: Large language models as collaborators for zero-shot medical reasoning," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 599–621.
13. X. Chen, H. Yi, M. You, W. Liu, L. Wang, H. Li, X. Zhang, Y. Guo, L. Fan, G. Chen, Q. Lao, W. Fu, K. Li, and J. Li, "Enhancing diagnostic capability with multi-agents conversational large language models," *npj Digital Medicine*, vol. 8, no. 1, p. 159, 03 2025. [Online]. Available: https://doi.org/10.1038/s41746-025-01550-0
14. T. Guarda, "Robotic process automation application in healthcare," in *Advanced Research in Technologies, Information, Innovation and Sustainability*, T. Guarda, F. Portela, and G. Gatica, Eds. Cham: Springer Nature Switzerland, 2025, pp. 97–109.
15. F. Huang and M. A. Vasarhelyi, "Applying robotic process automation (rpa) in auditing: A framework," *International Journal of Accounting Information Systems*, vol. 35, p. 100433, 2019.

16. Y. Li, Y. Yu, H. Li, Z. Chen, and K. Khashanah, "Tradinggpt: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance," 2023. [Online]. Available: https://arxiv.org/abs/2309.03736
17. Y. Wang, W. Zhou, Y. Li, and J. Sun, "Business optimization of financial centers in pharmaceutical enterprises based on robotic process automation technology," *IEEE Access*, vol. 13, pp. 51 012–51 026, 2025.
18. Y. Kim, I. Iturrate, C. Sloth, and H. Kim, "Safety-ensured control framework for robotic endoscopic task automation," 2025. [Online]. Available: https://arxiv.org/abs/2503.08214
19. T. Yao, M. Ban, B. Lu, Z. Pei, and P. Qi, "Sim4endor: A reinforcement learning centered simulation platform for task automation of endovascular robotics," 2025. [Online]. Available: https://arxiv.org/abs/2504.05330
20. J. Ruan, Y. Chen, B. Zhang, Z. Xu, T. Bao, du qing, shi shiwei, H. Mao, X. Zeng, and R. Zhao, "TPTU: Task planning and tool usage of large language model-based AI agents," in *NeurIPS 2023 Foundation Models for Decision Making Workshop*, 2023. [Online]. Available: https://openreview.net/forum?id=GrkgKtOjaH
21. Z. Liu, W. Yao, J. Zhang, L. Yang, Z. Liu, J. Tan, P. K. Choubey, T. Lan, J. Wu, H. Wang, S. Heinecke, C. Xiong, and S. Savarese, "Agentlite: A lightweight library for building and advancing task-oriented llm agent system," 2024. [Online]. Available: https://arxiv.org/abs/2402.15538
22. B. Y. Lin, C. Huang, Q. Liu, W. Gu, S. Sommerer, and X. Ren, "On grounded planning for embodied tasks with language models," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, pp. 13 192–13 200, Jun. 2023. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/26549
23. Y. Liang, C. Wu, T. Song, W. Wu, Y. Xia, Y. Liu, Y. Ou, S. Lu, L. Ji, S. Mao, Y. Wang, L. Shou, M. Gong, and N. Duan, "Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis," *Intelligent Computing*, vol. 3, p. 0063, 2024.
24. D. H. Autor, "Why are there still so many jobs? the history and future of workplace automation," *Journal of Economic Perspectives*, vol. 29, no. 3, pp. 3–30, September 2015. [Online]. Available: https://www.aeaweb.org/articles?id=10.1257/jep.29.3.3
25. A. D. Badmus, "Leveraging software automation to transform the manufacturing industry," *Journal of Knowledge Learning and Science Technology ISSN: 2959-6386 (online)*, vol. 2, no. 1, pp. 84–92, Jun. 2024. [Online]. Available: https://jklst.org/index.php/home/article/view/72
26. Y. Al-Slais and M. Ali, "Robotic process automation and intelligent automation security challenges: A review," in *2023 International Conference On Cyber Management And Engineering (CyMaEn)*, 2023, pp. 71–77.
27. R. Kavitha, "Hyperautomation-beyond rpa: : Leveraging automation to transform the manufacturing industries," in *2023 International Conference on Computer Communication and Informatics (ICCCI)*, 2023, pp. 1–5.
28. Airtable, "Airtable: The modern platform for building ai-powered apps," No-code database and application builder, 2025, accessed: 2024-11-12. [Online]. Available: https://www.airtable.com/
29. I. Figma, "Figma: The collaborative interface design tool," Online, 2025, accessed: 2025-02-24. [Online]. Available: https://www.figma.com/
30. Zapier, "Zapier: The most connected ai orchestration platform," 2024, accessed: 2024-10-24. [Online]. Available: https://zapier.com/
31. R. A. Team and Contributors, "Agentgpt: Autonomous ai agent platform," 2023, last accessed: 2025-02-09. [Online]. Available: https://github.com/reworkd/AgentGPT
32. N. Li, C. Gao, M. Li, Y. Li, and Q. Liao, "Econagent: Large language model-empowered agents for simulating macroeconomic activities," in *Proceedings of the 62nd Annual Meeting of the Association*

for Computational Linguistics (Volume 1: Long Papers). Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 15 523–15 536.
33. O. H. Abdellaif, A. Nader, and A. Hamdi, "Lmrpa: Large language model-driven efficient robotic process automation for ocr," 2024. [Online]. Available: https://arxiv.org/abs/2412.18063
34. C. JingXuan, M. Tayyab, S. M. Muzammal, N. Z. Jhanjhi, S. K. Ray, and F. Ashfaq, "Integrating ai with robotic process automation (rpa): Advancing intelligent automation systems," in *2024 IEEE 29th Asia Pacific Conference on Communications (APCC)*, 2024, pp. 259–265.
35. S. Erdmann and K. Sandkuhl, "Robotic process automation in small enterprises: An investigation into application potential," *Complex Systems Informatics and Modeling Quarterly*, vol. 34, pp. 84–105, 2023. [Online]. Available: https://doi.org/10.7250/csimq.2023-34.04
36. Botonic, "Botonic: No-code chatbots," No-code platform for building conversational chatbots, 2025, accessed: 2025-03-19. [Online]. Available: https://botonic.io/
37. Voiceflow, "Voiceflow: The fastest way to build advanced AI agents," No-code platform for building voice and chat agents, 2025, accessed: 2025-09-17. [Online]. Available: https://www.voiceflow.com/
38. Quickbase, "Quickbase: Spend more time on work that matters," 2024, quickbase helps you see, connect, and control your projects from day one to done - without the complexity. [Online]. Available: https://www.quickbase.com/
39. S. Yuan, K. Song, J. Chen, X. Tan, D. Li, and D. Yang, "Evoagent: Towards automatic multi-agent generation via evolutionary algorithms," in *Workshop on Open-World Agents: Synergizing Reasoning and Decision-Making in Open-World Environments (OWA-2024)*, 2024, poster Presentation. [Online]. Available: https://openreview.net/forum?id=DdxAp2j3AP
40. J. Riedesel, C. Myers, and B. Ashworth, "Intelligent space power automation," in *Proceedings. IEEE International Symposium on Intelligent Control 1989*, 1989, pp. 168–173.
41. Google, "Looker studio: No-code charts," Business intelligence and data visualization platform, 2025, accessed: 2025-04-08. [Online]. Available: https://lookerstudio.google.com/overview
42. Webnode, "Webnode: no-code website builder, make your own website for free!" Online, 2024, accessed: 2025-03-12. [Online]. Available: https://www.webnode.com
43. A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Technical Report, 2019, published on OpenAI Blog. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
44. G. Contributors, "Gpt researcher: Llm based autonomous agent that conducts deep local and web research on any topic and generates a long report with citations," 2024, accessed: 2024-08-29. [Online]. Available: https://github.com/assafelovic/gpt-researcher
45. OpenAI, :, A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney, A. Iftimie, A. Karpenko, A. T. Passos, A. Neitz, A. Prokofiev, A. Wei, A. Tam, and A. B. et al., "Openai o1 system card," 2024. [Online]. Available: https://arxiv.org/abs/2412.16720
46. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.
47. C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," 2024. [Online]. Available: https://arxiv.org/abs/2307.07924
48. D. Chen, H. Wang, Y. Huo, Y. Li, and H. Zhang, "Gamegpt: Multi-agent collaborative framework for game development," 2023. [Online]. Available: https://arxiv.org/abs/2310.08067
49. S. Wang, C. Liu, Z. Zheng, S. Qi, S. Chen, Q. Yang, A. Zhao, C. Wang, S. Song, and G. Huang,

"Avalon's game of thoughts: Battle against deception through recursive contemplation," 2023. [Online]. Available: https://arxiv.org/abs/2310.01320

50. Y. Xu, S. Wang, P. Li, F. Luo, X. Wang, W. Liu, and Y. Liu, "Exploring large language models for communication games: An empirical study on werewolf," 2024. [Online]. Available: https://arxiv.org/abs/2309.04658
51. D. Ramasubbu, K. Baskaran, and G. Yann, "Intrusive plug management system using chatbots in office environments," in *2018 Asian Conference on Energy, Power and Transportation Electrification (ACEPT)*, 2018, pp. 1–4.
52. D. R. Arikkat, A. M., N. Binu, P. M., N. Biju, K. S. Arunima, V. P., R. R. K. A., and M. Conti, "Intellbot: Retrieval augmented llm chatbot for cyber threat knowledge delivery," 2024. [Online]. Available: https://arxiv.org/abs/2411.05442
53. N. Livathinos, C. Auer, M. Lysak, A. Nassar, M. Dolfi, P. Vagenas, C. B. Ramis, and M. O. et al., "Docling: An efficient open-source toolkit for ai-driven document conversion," 2025. [Online]. Available: https://arxiv.org/abs/2501.17887
54. A. Perdana, W. E. Lee, and C. Mui Kim, "Prototyping and implementing robotic process automation in accounting firms: Benefits, challenges and opportunities to audit automation," *International Journal of Accounting Information Systems*, vol. 51, p. 100641, 2023.
55. A. Jain, S. Paliwal, M. Sharma, L. Vig, and G. Shroff, "Smartflow: Robotic process automation using llms," 2024. [Online]. Available: https://arxiv.org/abs/2405.12842
56. Z. Liu, W. Yao, J. Zhang, L. Xue, S. Heinecke, R. Murthy, Y. Feng, Z. Chen, J. C. Niebles, D. Arpit, R. Xu, P. Mui, H. Wang, C. Xiong, and S. Savarese, "Bolaa: Benchmarking and orchestrating llm-augmented autonomous agents," 2023. [Online]. Available: https://arxiv.org/abs/2308.05960
57. A. Bhatt and N. Vaghela, "Med-bot: An ai-powered assistant to provide accurate and reliable medical information," 2024. [Online]. Available: https://arxiv.org/abs/2411.09648
58. Y. Nam, T. Seo, G. Shin, S. Lee, and J. Im, "Novi : Chatbot system for university novice with bert and llms," 2024. [Online]. Available: https://arxiv.org/abs/2409.06192
59. Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "Toolllm: Facilitating large language models to master 16000+ real-world apis," 2023. [Online]. Available: https://arxiv.org/abs/2307.16789
60. S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VtmBAGCN7o
61. J. Duan, S. Wang, J. Diffenderfer, L. Sun, T. Chen, B. Kailkhura, and K. Xu, "ReTA: Recursively thinking ahead to improve the strategic reasoning of large language models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 2232–2246.
62. S. Mao, Y. Cai, Y. Xia, W. Wu, X. Wang, F. Wang, Q. Guan, T. Ge, and F. Wei, "ALYMPICS: LLM agents meet game theory," in *Proceedings of the 31st International Conference on Computational Linguistics*. Abu Dhabi, UAE: Association for Computational Linguistics, Jan. 2025, pp. 2845–2866. [Online]. Available: https://aclanthology.org/2025.coling-main.193/
63. P. Chen, S. Zhang, and B. Han, "CoMM: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving," in *Findings of the Association for Computational Linguistics: NAACL 2024*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 1720–1738.
64. W. Saeed and C. Omlin, "Explainable ai (xai): A systematic meta-survey of current challenges and future opportunities," *Knowledge-Based Systems*, vol. 263, p. 110273, 2023.

65. W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=EHg5GDnyq1
66. J. Wewerka and M. R. and, "Robotic process automation - a systematic mapping study and classification framework," *Enterprise Information Systems*, vol. 17, no. 2, p. 1986862, 2023.
67. W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, and K.-R. Muller, Eds., *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*, 1st ed., ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer-Verlag, 12 2022, vol. 11700.
68. Y. Kong, J. Ruan, Y. Chen, B. Zhang, T. Bao, S. Shiwei, d. G. Qing, X. Hu, H. Mao, Z. Li, X. Zeng, R. Zhao, and X. Wang, "Tptu-v2: Boosting task planning and tool usage of large language model-based agents in real-world industry systems," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Miami, Florida, US: Association for Computational Linguistics, Nov. 2024, pp. 371–385.
69. K. Lee, H. Zo, and M. Jeong, "Understanding user acceptance of robotic process automation: the user resistance perspective," *Industrial Management & Data Systems*, vol. 125, no. 3, pp. 1000–1022, 2025. [Online]. Available: https://doi.org/10.1108/IMDS-07-2024-0628
70. D. Schlegel, B. Rosenberg, O. Fundanovic, and P. Kraus, "How to conduct successful business process automation projects? an analysis of key factors in the context of robotic process automation," *Business Process Management Journal*, vol. 30, no. 8, pp. 99–119, 2024. [Online]. Available: https://doi.org/10.1108/BPMJ-06-2023-0465
71. M. A. Ferrag, N. Tihanyi, and M. Debbah, "From llm reasoning to autonomous ai agents: A comprehensive review," 2025. [Online]. Available: https://arxiv.org/abs/2504.19678
72. N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "Ctrl: A conditional transformer language model for controllable generation," 2019. [Online]. Available: https://arxiv.org/abs/1909.05858
73. T. Li, Y.-T. Yang, Y. Pan, and Q. Zhu, "From texts to shields: Convergence of large language models and cybersecurity," 2025. [Online]. Available: https://arxiv.org/abs/2505.00841
74. Z. Wang, S. Mao, W. Wu, T. Ge, F. Wei, and H. Ji, "Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 257–279. [Online]. Available: https://aclanthology.org/2024.naacl-long.15/
75. S. Afrin, S. Roksana, and R. Akram, "Ai-enhanced robotic process automation: A review of intelligent automation innovations," *IEEE Access*, vol. 13, pp. 173–197, 2025.
76. C.-H. Lin, D. K. W. Chiu, and K. T. Lam, "Hong kong academic librarians' attitudes toward robotic process automation," *Library Hi Tech*, vol. 42, no. 3, pp. 991–1014, 2024. [Online]. Available: https://doi.org/10.1108/LHT-03-2022-0141
77. X. Feng, L. Dou, E. Li, Q. Wang, H. Wang, Y. Guo, C. Ma, and L. Kong, "A survey on large language model-based social agents in game-theoretic scenarios," 2024. [Online]. Available: https://arxiv.org/abs/2412.03920
78. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," 2023. [Online]. Available: https://arxiv.org/abs/2210.03629
79. Z. Yang, L. Li, J. Wang, K. Lin, E. Azarnasab, F. Ahmed, Z. Liu, C. Liu, M. Zeng, and L. Wang, "Mm-react: Prompting chatgpt for multimodal reasoning and action," 2023. [Online]. Available: https://arxiv.org/abs/2303.11381
80. P. Prrucha and J. Skrbek, "Api as method for improving robotic process automation," in *Business*

Process Management: Blockchain, Robotic Process Automation, and Central and Eastern Europe Forum. Cham: Springer International Publishing, 2022, pp. 260–273.
81. Y. Asano, K. Okada, S. Nakagawa, N. Yoshie, and J. Shiomi, "Automation of polymer pressing by robotic handling with in-process parameter optimization," *Robotics and Autonomous Systems*, vol. 185, p. 104868, 2025.
82. J. G. Jacome-Leon, M. Zambrano-Vizuete, D. E. Imbaquingo-Esparza *et al.*, "Blockchain and machine learning for intelligent automation in robotic process automation," *SN Computer Science*, vol. 6, p. 341, 2025. [Online]. Available: https://doi.org/10.1007/s42979-025-03852-2
83. N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023. [Online]. Available: https://arxiv.org/abs/2303.11366
84. T. Vu, K. Krishna, S. Alzubi, C. Tar, M. Faruqui, and Y.-H. Sung, "Foundational autoraters: Taming large language models for better automatic evaluation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 17 086–17 105.
85. T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications," *IEEE Transactions on Cybernetics*, vol. 50, no. 9, pp. 3826–3839, 2020.
86. T. Masterman, S. Besen, M. Sawtell, and A. Chao, "The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2404.11584
87. T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, and A. H.-V. et al., "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165
88. H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. RoziÃ¨re, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971
89. H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, and L. B. et al., "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288
90. P. Zhou, J. Pujara, X. Ren, X. Chen, H.-T. Cheng, Q. V. Le, E. H. Chi, D. Zhou, S. Mishra, and H. S. Zheng, "Self-discover: Large language models self-compose reasoning structures," 2024. [Online]. Available: https://arxiv.org/abs/2402.03620
91. Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," 2023. [Online]. Available: https://arxiv.org/abs/2308.08155
92. X. Zhou, H. Zhu, L. Mathur, R. Zhang, H. Yu, Z. Qi, L.-P. Morency, Y. Bisk, D. Fried, G. Neubig, and M. Sap, "Sotopia: Interactive evaluation for social intelligence in language agents," 2024. [Online]. Available: https://arxiv.org/abs/2310.11667
93. J. Zhang, Y. Hou, R. Xie, W. Sun, J. McAuley, W. X. Zhao, L. Lin, and J.-R. Wen, "Agentcf: Collaborative learning with autonomous language agents for recommender systems," ser. WWW '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 3679–3689.
94. T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, Z. Liu, Y. Xu, H. SU, D. Shin, C. Xiong, and T. Yu, "Openagents: An open platform for language agents in the wild," in *First Conference on Language Modeling*, 2024. [Online]. Available: https://openreview.net/forum?id=sKATR2O1Y0
95. L. Weng, "Llm-powered autonomous agents," *Lilian Weng's Blog*, June 2023, accessed: 2025-02-09. [Online]. Available: https://lilianweng.github.io/posts/2023-06-23-agent/

96. S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: deliberate problem solving with large language models," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.
97. H. Xin, D. Guo, Z. Shao, Z. Ren, Q. Zhu, B. Liu, C. Ruan, W. Li, and X. Liang, "Advancing theorem proving in LLMs through large-scale synthetic data," in *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS'24*, 2024. [Online]. Available: https://openreview.net/forum?id=TPtXLihkny
98. H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, "Selfapr: Self-supervised program repair with test execution diagnostics," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3551349.3556926
99. J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=z5uVAKwmjf
100. Y. Ma, Z. Gou, J. Hao, R. Xu, S. Wang, L. Pan, Y. Yang, Y. Cao, and A. Sun, "SciAgent: Tool-augmented language models for scientific reasoning," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 15 701–15 736.
101. M. Luo, X. Xu, Y. Liu, P. Pasupat, and M. Kazemi, "In-context learning with retrieved demonstrations for language models: A survey," *Transactions on Machine Learning Research*, 2024, survey Certification. [Online]. Available: https://openreview.net/forum?id=NQPo8ZhQPa
102. N. Montes, M. Luck, N. Osman, O. Rodrigues, and C. Sierra, "Combining theory of mind and abductive reasoning in agent-oriented programming," in *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, ser. AAMAS '24. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2024, pp. 2839–2841.
103. A. Tiron-Tudor, R. Lacurezeanu, V. P. Bresfelean, and A. N. Dontu, "Perspectives on how robotic process automation is transforming accounting and auditing services," *Accounting Perspectives*, vol. 23, no. 1, pp. 7–38, 2024.
104. B. Axmann, S. A. Dzhelil, and A. Najeeb, "Practical comparison of uipath and power automate by creating an automation use case from logistics," in *2024 14th International Conference on Advanced Computer Information Technologies (ACIT)*, 2024, pp. 224–229.
105. S. Nithuna and C. Laseena, "Review on implementation techniques of chatbot," in *2020 International Conference on Communication and Signal Processing (ICCSP)*, 2020, pp. 0157–0161.
106. U. Inc., "The uipath platform: Where robots and agents unite," 2023, accessed: 2024-08-03. [Online]. Available: https://www.uipath.com/
107. L. Dobrica, "Robotic process automation platform uipath," *Commun. ACM*, vol. 65, no. 4, pp. 42–43, Mar. 2022. [Online]. Available: https://doi.org/10.1145/3511667
108. F. Corno, L. De Russis, and A. Monge Roffarello, "Heytap: Bridging the gaps between users' needs and technology in if-then rules via conversation," in *Proceedings of the 2020 International Conference on Advanced Visual Interfaces*, ser. AVI '20. New York, NY, USA: Association for Computing Machinery, 2020.
109. Y. Gao, K. Xiao, F. Li, W. Xu, J. Huang, and W. Dong, "Chatiot: Zero-code generation of trigger-action based iot programs," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 8, no. 3, Sep. 2024.
110. F. Corno, L. De Russis, and A. Monge Roffarello, "Recrules: Recommending if-then rules for end-user development," vol. 10, no. 5, Sep. 2019.
111. H. Chase, "Langchain: Large language model application development framework," Online,

2023, retrieved May 8, 2025, from official GitHub repository. [Online]. Available: https://github.com/langchain-ai/langchain
112. R. MacManus, "Langchain: The trendiest llm framework of 2023," *Tencent Cloud Developer Community*, 2023, published August 9, 2023; Retrieved May 8, 2025. [Online]. Available: https://cloud.tencent.com/developer/article/2309826
113. Torantulino, "Autogpt: An autonomous gpt-4 experiment," GitHub, 2023, accessed: 2025-05-13. [Online]. Available: https://github.com/Significant-Gravitas/Auto-GPT
114. H. Alps, "Openmanus: Open-source replication of manus ai agent," Online, 2025, retrieved May 8, 2025, from GitHub repository. [Online]. Available: https://github.com/henryalps/OpenManus
115. Y. Nakajima, "Babyagi: Autonomous self-building agent framework," 2024, last commit: 2024-04-15 (Accessed: 2025-05-09). [Online]. Available: https://github.com/yoheinakajima/babyagi
116. S. Contributors, "Superagi: A dev-first open source autonomous ai agent framework. enabling developers to build, manage & run useful autonomous agents quickly and reliably." 2024, accessed: 2024-04-24. [Online]. Available: https://github.com/TransformerOptimus/SuperAGI
117. M. Inc., "Manus ai: Autonomous general-purpose ai agent," Online, 2025, retrieved May 8, 2025, from official website. [Online]. Available: https://manus.im/
118. M. Goyal and P. Bhasin, "Beyond the model: Key differentiators in large language models and multi-agent services," *World Journal of Advanced Research and Reviews*, vol. 26, no. 01, pp. 2703–2706, 2025, review Article. [Online]. Available: https://doi.org/10.30574/wjarr.2025.26.1.1295
119. M. Shen and Q. Yang, "From mind to machine: The rise of manus ai as a fully autonomous digital agent," 2025. [Online]. Available: https://arxiv.org/abs/2505.02024
120. CAMEL-AI, "Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation," GitHub, 2024, accessed: [2024-07-15]. [Online]. Available: https://github.com/camel-ai/owl
121. P. Venkadesh, S. V. Divya, and K. S. Kumar, "Unlocking ai creativity: A multi-agent approach with crewai," *Journal of Trends in Computer Science and Smart Technology*, vol. 6, no. 4, pp. 338–356, 2024.
122. AirSlate, "airslate: Automate document workflow," airSlate Inc., 2024, accessed: 2025-05-18. [Online]. Available: https://www.airslate.com/
123. J. Tang, T. Fan, and C. Huang, "Autoagent: A fully-automated and zero-code framework for llm agents," 2025. [Online]. Available: https://arxiv.org/abs/2502.05957
124. Y. Feng, S. Papicchio, and S. Rahman, "Cypherbench: Towards precise retrieval over full-scale modern knowledge graphs in the llm era," 2025. [Online]. Available: https://arxiv.org/abs/2412.18702
125. S. Sukhbaatar, A. Szlam, and R. Fergus, "Learning multiagent communication with backpropagation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 2244–2252. [Online]. Available: https://proceedings.neurips.cc/paper/2016/file/55b1927fdafef39c48e5b73b5d61ea60-Paper.pdf
126. B. Pan, J. Lu, K. Wang, L. Zheng, Z. Wen, Y. Feng, M. Zhu, and W. Chen, "Agentcoord: Visually exploring coordination strategy for llm-based multi-agent collaboration," 2024. [Online]. Available: https://arxiv.org/abs/2404.11943
127. S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems," 2025. [Online]. Available: https://arxiv.org/abs/2505.00212
128. M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, "API-bank: A comprehensive benchmark for tool-augmented LLMs," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 3102–3116.
129. T. Liu, X. Wang, W. Huang, W. Xu, Y. Zeng, L. Jiang, H. Yang, and J. Li, "Groupdebate:

Enhancing the efficiency of multi-agent debate using group discussion," 2024. [Online]. Available: https://arxiv.org/abs/2409.14051
130. D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "Agentcoder: Multi-agent-based code generation with iterative testing and optimisation," 2024. [Online]. Available: https://arxiv.org/abs/2312.13010
131. A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, and G. L. et al., "Mixtral of experts," 2024. [Online]. Available: https://arxiv.org/abs/2401.04088
132. Y. Ge, W. Hua, K. Mei, J. Ji, J. Tan, S. Xu, Z. Li, and Y. Zhang, "Openagi: when llm meets domain experts," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.
133. B. Ni and M. J. Buehler, "Mechagents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge," *Extreme Mechanics Letters*, vol. 67, p. 102131, 2024.
134. J. Chen, X. Hu, S. Liu, S. Huang, W.-W. Tu, Z. He, and L. Wen, "Llmarena: Assessing capabilities of large language models in dynamic multi-agent environments," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 13 055–13 077.
135. L. Cross, V. Xiang, A. Bhatia, D. L. Yamins, and N. Haber, "Hypothetical minds: Scaffolding theory of mind for multi-agent tasks with large language models," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=otW0TJOUYF
136. J. Owotogbe, "Assessing and enhancing the robustness of llm-based multi-agent systems through chaos engineering," 2025. [Online]. Available: https://arxiv.org/abs/2505.03096
137. B. Ur, M. Pak Yong Ho, S. Brawner, J. Lee, S. Mennicken, N. Picard, D. Schulze, and M. L. Littman, "Trigger-action programming in the wild: An analysis of 200,000 ifttt recipes," in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, ser. CHI '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 3227–3231.
138. T. Fan, J. Wang, X. Ren, and C. Huang, "Minirag: Towards extremely simple retrieval-augmented generation," *arXiv preprint arXiv:2501.06713*, 2025.
139. A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," 2023. [Online]. Available: https://arxiv.org/abs/2310.11511
140. N. M, "Deeprag: Building a custom hindi embedding model for retrieval augmented generation from scratch," 2025. [Online]. Available: https://arxiv.org/abs/2503.08213
141. M. Besta, J. Barth, E. Schreiber, A. Kubicek, A. Catarino, R. Gerstenberger, P. Nyczyk, P. Iff, Y. Li, S. Houliston, T. Sternal, M. Copik, G. Kwasniewski, J. MÃ¼ller, L. Flis, H. Eberhard, H. Niewiadomski, and T. Hoefler, "Reasoning language models: A blueprint," 2025. [Online]. Available: https://arxiv.org/abs/2501.11223
142. K. Luo, Z. Ding, Z. Weng, L. Qiao, M. Zhao, X. Li, di yin, and J. Shu, "Let's be self-generated via step by step: A curriculum learning approach to automated reasoning with large language models," 2025. [Online]. Available: https://openreview.net/forum?id=ixoIAOcTSx
143. J. Muller and J. Oupicky, "Post-quantum XML and SAML single sign-on," *Proceedings on Privacy Enhancing Technologies*, vol. 2024, no. 4, pp. 525–543, 2024, pOPETS 2024, Issue 4. [Online]. Available: https://doi.org/10.56553/popets-2024-0128
144. S. Bruch, S. Gai, and A. Ingber, "An analysis of fusion functions for hybrid retrieval," *ACM Transactions on Information Systems*, vol. 42, no. 1, pp. 1–35, Aug. 2023. [Online]. Available: http://dx.doi.org/10.1145/3596512
145. N. Rajkumar, R. Li, and D. Bahdanau, "Evaluating the text-to-sql capabilities of large language models,"

2022. [Online]. Available: https://arxiv.org/abs/2204.00498
146. J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," 2024. [Online]. Available: https://arxiv.org/abs/2406.04692
147. W. Zhou, Y. E. Jiang, L. Li, J. Wu, T. Wang, S. Qiu, J. Zhang, J. Chen, R. Wu, S. Wang, S. Zhu, J. Chen, W. Zhang, X. Tang, N. Zhang, H. Chen, P. Cui, and M. Sachan, "Agents: An open-source framework for autonomous language agents," 2023. [Online]. Available: https://arxiv.org/abs/2309.07870
148. W. Yao, S. Heinecke, J. C. Niebles, Z. Liu, Y. Feng, L. Xue, R. R. N, Z. Chen, J. Zhang, D. Arpit, R. Xu, P. L. Mui, H. Wang, C. Xiong, and S. Savarese, "Retroformer: Retrospective large language agents with policy gradient optimization," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=KOZu91CzbK
149. W. Jin, B. Zhao, H. Yu, X. Tao, R. Yin, and G. Liu, "Improving embedded knowledge graph multi-hop question answering by introducing relational chain reasoning," *Data Mining and Knowledge Discovery*, vol. 37, no. 1, pp. 255–288, 2023. [Online]. Available: https://doi.org/10.1007/s10618-022-00891-8
150. K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "Colbertv2: Effective and efficient retrieval via lightweight late interaction," 2022. [Online]. Available: https://arxiv.org/abs/2112.01488
151. J. Chen, A. Prasad, S. Saha, E. Stengel-Eskin, and M. Bansal, "MAgICore: Multi-agent, iterative, coarse-to-fine refinement for reasoning," 2025. [Online]. Available: https://openreview.net/forum?id=j9wBgcxa7N
152. Z. Wang, J. Liu, S. Zhang, and Y. Yang, "Poisoned langchain: Jailbreak llms by langchain," 2024. [Online]. Available: https://arxiv.org/abs/2406.18122
153. P. Lu, B. Chen, S. Liu, R. Thapa, J. Boen, and J. Zou, "Octotools: An agentic framework with extensible tools for complex reasoning," 2025. [Online]. Available: https://arxiv.org/abs/2502.11271
154. Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen, "Tora: A tool-integrated reasoning agent for mathematical problem solving," 2024. [Online]. Available: https://arxiv.org/abs/2309.17452
155. C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, "Llm-planner: Few-shot grounded planning for embodied agents with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2212.04088
156. A. Radford, J. Wu, R. Child *et al.*, "Better language models and their implications," OpenAI Blog, 2019. [Online]. Available: https://openai.com/research/better-language-models
157. DeepAI, "Deepai: Artificial intelligence tools and services," 2025, accessed: 2025-04-03. [Online]. Available: https://deepai.org/
158. I. Hugging Face, "Hugging face: The ai community building the future." accessed: 2025-03-13. [Online]. Available: https://huggingface.co
159. Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 38 154–38 180. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/77c33e6a367922d003ff102ffb92b658-Paper-Conference.pdf
160. A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.
161. OpenAI, "Whisper: Robust speech recognition via large-scale weak supervision," 2022, accessed: 2025-05-13. [Online]. Available: https://github.com/openai/whisper
162. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, ser.

Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 8748–8763, oral presentation at ICML 2021. [Online]. Available: https://icml.cc/virtual/2021/oral/9194
163. I. Alshehri, A. Alshehri, A. Almalki, M. Bamardouf, and A. Akbar, "Breachseek: A multi-agent automated penetration tester," 2024. [Online]. Available: https://arxiv.org/abs/2409.03789
164. B. Khemani, S. Patil, K. Kotecha, and S. Tanwar, "A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions," *Journal of Big Data*, vol. 11, no. 1, p. 18, 2024.
165. M. Jiang, G. Liu, B. Zhao, Y. Su, and W. Jin, "Relation-aware graph structure embedding with co-contrastive learning for drug-drug interaction prediction," *Neurocomputing*, vol. 572, p. 127203, 2024.
166. M. Jiang, G. Liu, Y. Su, W. Jin, and B. Zhao, "Hierarchical multi-relational graph representation learning for large-scale prediction of drug-drug interactions," *IEEE Transactions on Big Data*, pp. 1–14, 2025.
167. X. Peng, Y. Liu, Y. Cang, C. Cao, and M. Chen, "Llm-optira: Llm-driven optimization of resource allocation for non-convex problems in wireless communications," 2025. [Online]. Available: https://arxiv.org/abs/2505.02091
168. D. Marta, S. Holk, M. Vasco, J. Lundell, T. Homberger, F. Busch, O. Andersson, D. Kragic, and I. Leite, "Flora: Sample-efficient preference-based rl via low-rank style adaptation of reward functions," 2025. [Online]. Available: https://arxiv.org/abs/2504.10002
169. W. Jin, B. Zhao, L. Zhang, C. Liu, and H. Yu, "Back to common sense: Oxford dictionary descriptive knowledge augmentation for aspect-based sentiment analysis," *Information Processing & Management*, vol. 60, no. 3, p. 103260, 2023.
170. V. Yadav and S. Bethard, "A survey on recent advances in named entity recognition from deep learning models," in *Proceedings of the 27th International Conference on Computational Linguistics*, E. M. Bender, L. Derczynski, and P. Isabelle, Eds. Santa Fe, New Mexico, USA: Association for Computational Linguistics, Aug. 2018, pp. 2145–2158. [Online]. Available: https://aclanthology.org/C18-1182/
171. T. Kudo, "Subword regularization: Improving neural network translation models with multiple subword candidates," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 66–75.
172. B. Wong and K. Tanaka, "High-fidelity pseudo-label generation by large language models for training robust radiology report classifiers," 2025. [Online]. Available: https://arxiv.org/abs/2505.01693
173. W. Jin, B. Zhao, Y. Zhang, J. Huang, and H. Yu, "Wordtransabsa: Enhancing aspect-based sentiment analysis with masked language modeling for affective token prediction," *Expert Systems with Applications*, vol. 238, p. 122289, 2024.
174. Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299.
175. H. Li, Y. Chong, S. Stepputtis, J. Campbell, D. Hughes, C. Lewis, and K. Sycara, "Theory of mind for multi-agent collaboration via large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 180–192.
176. K. Nottingham, P. Ammanabrolu, A. Suhr, Y. Choi, H. Hajishirzi, S. Singh, and R. Fox, "Do embodied agents dream of pixelated sheep?: Embodied decision making using language guided world modelling," in *Workshop on Reincarnating Reinforcement Learning at ICLR 2023*, 2023. [Online]. Available: https://openreview.net/forum?id=Z_qiOvqvnBl
177. M. Vidgof, S. Bachhofner, and J. Mendling, "Large language models for business process management: Opportunities and challenges," in *Business Process Management Forum*, C. Di Francescomarino,

A. Burattin, C. Janiesch, and S. Sadiq, Eds. Cham: Springer Nature Switzerland, 2023, pp. 107–123.
178. J. Haase, W. Kremser, H. Leopold, J. Mendling, L. Onnasch, and R. Plattfaut, "Interdisciplinary directions for researching the effects of robotic process automation and large language models on business processes," *Communications of the Association for Information Systems*, vol. 54, pp. 579–604, 2024. [Online]. Available: https://doi.org/10.17705/1CAIS.05421
179. Z. Kaiya, M. Naim, J. Kondic, M. Cortes, J. Ge, S. Luo, G. R. Yang, and A. Ahn, "Lyfe agents: Generative agents for low-cost real-time social interactions," 2023. [Online]. Available: https://arxiv.org/abs/2310.02172
180. N. Frick, "Barriers, facilitators and prerequisites for robotic process automation adoption in public administrations: A systematic literature review," in *Proceedings of the European Conference on Information Systems (ECIS 2024)*. ECIS, 2024, complete Research Paper. Paper 6. [Online]. Available: https://aisel.aisnet.org/ecis2024_rp/6
181. J. Primbs, D. Kern, M. Menth, and C. KrauB, "Streamlining plug-and-charge authorization for electric vehicles with oauth2 and oidc," 2025. [Online]. Available: https://arxiv.org/abs/2501.14397
182. T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best current practice for oauth 2.0 security," RFC 9700, Jan. 2025. [Online]. Available: https://www.rfc-editor.org/info/rfc9700
183. Z. Hartl and A. Derek, "Towards automated formal security analysis of saml v2.0 web browser sso standard – the post/artifact use case," 2024. [Online]. Available: https://arxiv.org/abs/2403.11859
184. Y. Zhang, "New approaches to automated software testing based on artificial intelligence," in *2024 5th International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, 2024, pp. 806–810.
185. P. Pham, V. Nguyen, and T. Nguyen, "A review of ai-augmented end-to-end test automation tools," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: Association for Computing Machinery, 2023.
186. F. Ricca, A. Marchetto, and A. Stocco, "Ai-based test automation: A grey literature analysis," in *2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2021, pp. 263–270.
187. B. Xiao, Z. Yin, and Z. Shan, "Simulating public administration crisis: A novel generative agent-based simulation system to lower technology barriers in social science research," 2023. [Online]. Available: https://arxiv.org/abs/2311.06957
188. S. Singh and J. Heard, "Human-aware reinforcement learning for adaptive human robot teaming," in *2022 17th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, 2022, pp. 1049–1052.
189. N. Amirshirzad, A. Kumru, and E. Oztop, "Human adaptation to human-robot shared control," *IEEE Transactions on Human-Machine Systems*, vol. 49, no. 2, pp. 126–136, 2019.
190. D. S. Lange and R. S. Gutzwiller, "Human-autonomy teaming patterns in the command and control of teams of autonomous systems," in *Engineering Psychology and Cognitive Ergonomics*. Cham: Springer International Publishing, 2016, pp. 179–188.
191. S. Holt, M. R. Luyten, and M. van der Schaar, "L2mac: Large language model automatic computer for extensive code generation," 2024. [Online]. Available: https://arxiv.org/abs/2310.02003
192. Eumemic and Contributors, "Ai legion: An llm-powered autonomous agent platform," 2023, last commit: 2024-02-15 (Accessed: 2025-05-09). [Online]. Available: https://github.com/eumemic/ai-legion
193. L. Xu, Z. Hu, D. Zhou, H. Ren, Z. Dong, K. Keutzer, S.-K. Ng, and J. Feng, "Magic: Investigation of large language model powered multi-agent in cognition, adaptability, rationality and collaboration," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 7315–7332.
194. T. Ridnik, D. Kredo, and I. Friedman, "Code generation with alphacodium: From prompt engineering

to flow engineering," 2024. [Online]. Available: https://arxiv.org/abs/2401.08500
195. F. Rahman, "Loopgpt: Modular auto-gpt framework," 2023, accessed: 2025-01-02. [Online]. Available: https://pypi.org/project/loopgpt/
196. A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: Llm agents are experiential learners," ser. AAAI'24/IAAI'24/EAAI'24. AAAI Press, 2024.
197. J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser, "Tidybot: Personalized robot assistance with large language models," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 3546–3553.
198. L. G. Mugnaini, B. L. Yamamoto, L. L. de Alcantara, V. Zacarias, E. Bollis, L. Pellicer, A. H. R. Costa, and A. Jordao, "Efficient llms with amp: Attention heads and mlp pruning," 2025. [Online]. Available: https://arxiv.org/abs/2504.21174
199. OpenAI, "Gpt-4 technical report," arXiv:2303.08774, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774
200. C. R. Team, "Cogito: A hybrid reasoning model with iterative distillation and amplification," *Technical Report*, 2023, proprietary model combining symbolic reasoning and neural networks. [Online]. Available: https://www.cogitocorp.com
201. OpenAI, "Introducing GPT-4.5," https://openai.com/index/introducing-gpt-4-5/, 2025, accessed: 2024-12-04.
202. Y. Dan, Z. Lei, Y. Gu, Y. Li, J. Yin, J. Lin, L. Ye, Z. Tie, Y. Zhou, Y. Wang, A. Zhou, Z. Zhou, Q. Chen, J. Zhou, L. He, and X. Qiu, "Educhat: A large language model-based conversational agent for intelligent education," in *China Conference on Knowledge Graph and Semantic Computing and International Joint Conference on Knowledge Graphs*. Singapore: Springer Nature Singapore, 2025, pp. 297–308.
203. T. Wang, N. Zhou, and Z. Chen, "Cybermentor: Ai powered learning tool platform to address diverse student needs in cybersecurity education," 2025. [Online]. Available: https://arxiv.org/abs/2501.09709
204. C. Gao, X. Lan, Z. Lu, J. Mao, J. Piao, H. Wang, D. Jin, and Y. Li, "S3: Social-network simulation system with large language model-empowered agents," 2023. [Online]. Available: https://arxiv.org/abs/2307.14984
205. Y. Wang, Z. Jiang, Z. Chen, F. Yang, Y. Zhou, E. Cho, X. Fan, Y. Lu, X. Huang, and Y. Yang, "Recmind: Large language model powered agent for recommendation," in *Findings of the Association for Computational Linguistics: NAACL 2024*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 4351–4364.
206. E. Tzanis and M. E. Klontzas, "maistro: an open-source multi-agentic system for automated end-to-end development of radiomics and deep learning models for medical imaging," 2025. [Online]. Available: https://arxiv.org/abs/2505.03785
207. D. GmbH, "Deepl translate: The world's most accurate translator," Maastricht, Netherlands, 2023, online machine translation service, Accessed: 2025-09-09. [Online]. Available: https://www.deepl.com/translator
208. Z. Rackauckas, "Rag-fusion: A new take on retrieval augmented generation," *International Journal on Natural Language Computing*, vol. 13, no. 1, pp. 37–47, Feb. 2024. [Online]. Available: http://dx.doi.org/10.5121/ijnlc.2024.13103
209. Y. P. Tsang, V. Tang, C. H. Wu, and F. Li, "Unlocking the potential of robotic process automation for digital transformation in logistics and supply chain management," *Journal of Global Information Management*, vol. 32, no. 1, p. 24, 2024. [Online]. Available: https://doi.org/10.4018/JGIM.361710
210. Q. Zhao, J. Wang, Y. Zhang, Y. Jin, K. Zhu, H. Chen, and X. Xie, "Competeai: understanding the competition dynamics of large language model-based agents," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
211. F. A. Lievano-MartÃnez, J. D. FernÃ¡ndez-Ledesma, D. Burgos, J. W. Branch-Bedoya, and

J. A. Jimenez-Builes, "Intelligent process automation: An application in manufacturing industry," *Sustainability*, vol. 14, no. 14, 2022. [Online]. Available: https://www.mdpi.com/2071-1050/14/14/8804
212. Y. Zhang, S. Hao, and S. Li, "Development and application of electronic tax declaration robot for individual income tax based on uipath," in *Proceedings of the 2023 3rd International Conference on Big Data, Artificial Intelligence and Risk Management*, ser. ICBAR '23. New York, NY, USA: Association for Computing Machinery, 2024, pp. 620–624.
213. M. Weiss, N. Rahaman, M. Wuthrich, Y. Bengio, L. E. Li, B. Schölkopf, and C. Pal, "Rethinking the buyer's inspection paradox in information markets with language agents," 2024. [Online]. Available: https://openreview.net/forum?id=6werMQy1uz
214. O. B. Akintuyi, "Ai in agriculture: A comparative review of developments in the usa and africa," *Open Access Research Journal of Science and Technology*, vol. 10, no. 2, pp. 060–070, 2024.
215. X. Zhou, G. Li, Z. Sun, Z. Liu, W. Chen, J. Wu, J. Liu, R. Feng, and G. Zeng, "D-bot: Database diagnosis system using large language models," *Proc. VLDB Endow.*, vol. 17, no. 10, pp. 2514–2527, Jun. 2024. [Online]. Available: https://doi.org/10.14778/3675034.3675043
216. M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. MercangÃ¶z, "Leveraging llm agents and digital twins for fault handling in process plants," 2025. [Online]. Available: https://arxiv.org/abs/2505.02076
217. O. Team and Contributors, "Workgpt: Autonomous agent framework for api orchestration," 2024, accessed: 2025-03-09. [Online]. Available: https://www.npmjs.com/package/workgpt
218. BD (Becton, Dickinson and Company), *HemoSphere Alta™ Monitor: Hemodynamic monitoring system with predictive analytics and multi-sensor integration*, 1st ed., Franklin Lakes, NJ, USA, 2023. [Online]. Available: https://www.bd.com/en-us/products-and-solutions/products/product-families/hemosphere-alta-monitor
219. M. Z. Raza, J. Xu, T. Lim, L. Boddy, C. M. Mery, A. Well, and Y. Ding, "Llm-ta: An llm-enhanced thematic analysis pipeline for transcripts from parents of children with congenital heart disease," 2025. [Online]. Available: https://arxiv.org/abs/2502.01620
220. Cincoze Co., Ltd., "Cincoze edge ai," New Taipei City, Taiwan, 2024, accessed: 2025-02-23. [Online]. Available: https://www.cincoze.com/zh-tw/
221. A. Yaacoub, J. Da-Rugna, and Z. Assaghir, "Assessing ai-generated questions' alignment with cognitive frameworks in educational assessment," 2025. [Online]. Available: https://arxiv.org/abs/2504.14232
222. OneClickQuiz, "Boost moodle with ai-driven quizzes aligned to cognitive & linguistic standards," 2023, accessed: [2024-06-01]. [Online]. Available: https://linktr.ee/oneclickquiz
223. S. Kaiser, A. Å. Tosun, and T. Korkmaz, "Benchmarking container technologies on arm-based edge devices," *IEEE Access*, vol. 11, pp. 107 331–107 347, 2023.
224. S. Kaiser, M. S. Haq, A. Å. Tosun, and T. Korkmaz, "Container technologies for arm architecture: A comprehensive survey of the state-of-the-art," *IEEE Access*, vol. 10, pp. 84 853–84 881, 2022.
225. Q. Fang, Y. Zhou, S. Guo, S. Zhang, and Y. Feng, "Llama-omni2: Llm-based real-time spoken chatbot with autoregressive streaming speech synthesis," 2025. [Online]. Available: https://arxiv.org/abs/2505.02625
226. Tray.AI, "Tray.ai: Ai-ready integration & automation platform," 2023, accessed: 2023-11-15. [Online]. Available: https://tray.ai
227. Y. Li, L. Sun, and Y. Zhang, "Metaagents: Large language model based agents for decision-making on teaming," *Proc. ACM Hum.-Comput. Interact.*, vol. 9, no. 2, May 2025. [Online]. Available: https://doi.org/10.1145/3711032
228. LangChain Inc., "Langgraph: Stateful, multi-agent llm application framework," Online, 2024, retrieved May 8, 2025. [Online]. Available: https://github.com/langchain-ai/langgraph
229. L. M. V. da Silva, A. KÃ¶cher, N. KÃ¶nig, F. Gehlhoff, and A. Fay, "Capability-driven skill generation with llms: A rag-based approach for reusing existing libraries and interfaces," 2025. [Online]. Available:

https://arxiv.org/abs/2505.03295
230. L. Luo, Y. Liu, R. Liu, S. Phatale, M. Guo, H. Lara, Y. Li, L. Shu, L. Meng, J. Sun, and A. Rastogi, "Improve mathematical reasoning in language models with automated process supervision," 2025. [Online]. Available: https://openreview.net/forum?id=KwPUQOQIKt
231. Chroma, "Chroma: The open-source ai application database," 2023, accessed: 2023-11-15. [Online]. Available: https://docs.trychroma.com/
232. K. A. Fischer, "Reflective linguistic programming (rlp): A stepping stone in socially-aware agi (socialagi)," 2023. [Online]. Available: https://arxiv.org/abs/2305.12647
233. Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via chatgpt," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, Sep. 2024.
234. N. Ashrafi, S. Bouktif, and M. Mediani, "Enhancing llm code generation: A systematic evaluation of multi-agent collaboration and runtime debugging for improved accuracy, reliability, and latency," 2025. [Online]. Available: https://arxiv.org/abs/2505.02133
235. H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, "Building cooperative embodied agents modularly with large language models," 2024. [Online]. Available: https://arxiv.org/abs/2307.02485
236. G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "Gaia: a benchmark for general ai assistants," 2023. [Online]. Available: https://arxiv.org/abs/2311.12983
237. T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 68 539–68 551. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf
238. C. Inc., "Crewai: Framework for orchestrating role-playing, autonomous ai agents," Online, 2025, retrieved May 8, 2025, from official GitHub repository. [Online]. Available: https://github.com/crewAIInc/crewAI
239. aisuhua, "Restful api design references," 2023, accessed: 2023-11-05. [Online]. Available: https://github.com/aisuhua/restful-api-design-references
240. J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
241. J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu *et al.*, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data*. ACM, 2021, pp. 1–13.
242. LangChain, "Langchain expression language (lcel)," 2023, accessed: 2023-05-15. [Online]. Available: https://python.langchain.com/docs/concepts/lcel/
243. J. Wang and Z. Duan, "Agent ai with langgraph: A modular framework for enhancing machine translation using large language models," 2024. [Online]. Available: https://arxiv.org/abs/2412.03801
244. J. Liu, "Llamaindex: Data framework for large language model applications," Online, 2022, retrieved May 8, 2025, from official GitHub repository. [Online]. Available: https://github.com/run-llama/llama_index
245. A. M. Radke, M. T. Dang, and A. Tan, "Using robotic process automation (rpa) to enhance item master data maintenance process," *Logforum*, vol. 16, no. 1, p. 10, 2020.
246. Forethought, "Autochain: Building generative agents with large language models," 2023, accessed: 2023-11-15. [Online]. Available: https://autochain.forethought.ai/
247. C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, "Appagent: Multimodal agents as smartphone users," in *Proceedings of the 2025 CHI Conference on Human Factors in*

Computing Systems, ser. CHI '25. New York, NY, USA: Association for Computing Machinery, 2025.
248. B. Y. Lin, Y. Fu, K. Yang, F. Brahman, S. Huang, C. Bhagavatula, P. Ammanabrolu, Y. Choi, and X. Ren, "Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 23 813–23 825. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/4b0eea69deea512c9e2c469187643dc2-Paper-Conference.pdf
249. FlowiseAI, "Flowise: Open source low-code tool for building llm apps," 2023, accessed: 2023-11-15. [Online]. Available: https://flowiseai.com/
250. Retool, "Retool: The best way to build internal software," 2025, accessed: 2025-01-09. [Online]. Available: https://retool.com/
251. Z. Rasheed, M. Waseem, A. Ahmad, K.-K. Kemell, W. Xiaofeng, A. N. Duc, and P. Abrahamsson, "Can large language models serve as data analysts? a multi-agent assisted approach for qualitative data analysis," 2024. [Online]. Available: https://arxiv.org/abs/2402.01386
252. A. Smith and J. Anderson, "Ai, robotics, and the future of jobs," Pew Research Center, Research Report, 08 2014, accessed: 2024-06-30. [Online]. Available: https://www.pewresearch.org/internet/2014/08/06/future-of-jobs/
253. A. Setlur, C. Nagpal, A. Fisch, X. Geng, J. Eisenstein, R. Agarwal, A. Agarwal, J. Berant, and A. Kumar, "Rewarding progress: Scaling automated process verifiers for LLM reasoning," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=A6Y7AqlzLW
254. N. Nascimento, P. Alencar, and D. Cowan, "Gpt-in-the-loop: Adaptive decision-making for multiagent systems," 2023. [Online]. Available: https://arxiv.org/abs/2308.10435
255. T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, "Encouraging divergent thinking in large language models through multi-agent debate," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 17 889–17 904. [Online]. Available: https://aclanthology.org/2024.emnlp-main.992/
256. Y. Wu, S. Y. Min, Y. Bisk, R. Salakhutdinov, A. Azaria, Y. Li, T. Mitchell, and S. Prabhumoye, "Plan, eliminate, and track – language models are good teachers for embodied agents," 2023. [Online]. Available: https://arxiv.org/abs/2305.02412
257. G. Kovac, R. Portelas, P. F. Dominey, and P.-Y. Oudeyer, "The socialai school: a framework leveraging developmental psychology toward artificial socio-cultural agents," *Frontiers in Neurorobotics*, vol. Volume 18 - 2024, 2024.
258. T. Y. Zhuo, Z. Li, Y. Huang, F. Shiri, W. Wang, G. Haffari, and Y.-F. Li, "On robustness of prompt-based semantic parsing with large pre-trained language model: An empirical study on codex," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. Dubrovnik, Croatia: Association for Computational Linguistics, May 2023, pp. 1090–1102.